

vuln_lkm_run_debug.pdf

Install, debug, and theory: character devices, ioctl, IRQs, overflows

Repository: github.com/mayday5000/Vuln-LKM

QEMU GDB stub: tcp::1234 (scripts/run_qemu.sh)

Breakpoints: vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str

Educational lab — not an exploit guide. Target length: 40+ pages.

Educational material for the Vuln-LKM loadable kernel module. Not an exploit guide. Integer wrap and a bounded-struct string copy are described as they appear in the source.

Contents

1. What this manual is
2. Character devices on Linux
3. ioctl(2): command words, copy_*_user, unlocked_ioctl
4. IRQs (Windows) and the Linux equivalents
5. Integer overflow and underflow
6. Buffer overflows in kernel C
7. Install: headers, QEMU, GDB
8. Host build and load
9. QEMU lab: expected serial output
10. GDB remote: expected session
11. Step-by-step: GET / SET / ADD / SUB / STR
12. Mapping CLI to kernel and breakpoints
- Appendix A. Linux driver model (survey)
- Appendix B. Kernel map for this lab
- Appendix C. Full source and scripts
- Appendix D. Glossary
- Appendix E. Failure table and lab rules

1. What this manual is

This is the install-and-debug companion to `vuln_lkm.pdf` (how the module works) and `vuln_lkm_cli.pdf` (how the tester works). It is written so you can take a clean Linux box or WSL userspace, install QEMU and GDB, boot a guest with a GDB stub, load `vuln_lkm.ko`, and step the three intentional bugs without guessing command lines.

It is also a theory manual. Character devices, `ioctl` command packing, `copy_from_user`, Windows IRQLs versus Linux process/softirq/hardirq context, unsigned wrap, and stack memcpy are the ideas the lab is for. The QEMU and GDB blocks are labeled as expected lab sessions: they follow the scripts and `printk` format in the repo. They are not a claim that a particular host captured that exact transcript today.

Vuln-LKM is a lab, not an exploit kit. This PDF does not include privilege-escalation payloads, SMEP or KASLR bypasses, or a weaponized overflow. STR with len greater than 32 can oops a kernel. Do that in QEMU. Do not load this module on a machine you cannot afford to reboot.

1.1 Objects in the repo

- `vuln_lkm.c` / `vuln_lkm.h` — out-of-tree LKM, misc character device `/dev/vuln_lkm`
- `vuln_lkm_cli.c` — userspace `ioctl` tester (interactive + argv)
- `Makefile` — `kbuild` for the `.ko`, `gcc` for the CLI
- `load.sh` / `unload.sh` — `insmod`/`rmmod` helpers
- `scripts/install_qemu_gdb.sh` — distro packages for QEMU and GDB
- `scripts/run_qemu.sh` — `qemu-system-x86_64`, `-gdb tcp::1234`, `nokaslr`, `9p modshare`
- `scripts/debug_gdb.sh` + `scripts/vuln_lkm.gdb` — remote GDB and four breakpoints

1.2 What success looks like

On a host kernel you care about: `make`, `./load.sh`, `./vuln_lkm_cli` get prints 100, SET/ADD/SUB wrap as documented, you `rmmod` when done. You skip long STR. In QEMU: the guest boots with a serial console, you mount the `9p` share or copy the `.ko`, `insmod`, attach GDB, break in `vuln_lkm_add` / `vuln_lkm_sub` / `vuln_lkm_str`, print `g_val` and `kbuf`, continue. After an oops you reboot the guest, not your laptop.

2. Character devices on Linux

Unix split block devices (random access, request queues, later the block layer) from character devices (byte streams, or more honestly: anything that is not a block device). A character device is identified by a (major, minor) pair. The major selects a driver. The minor selects an instance. Userspace sees this as a special inode: `ls -l /dev/vuln_lkm` shows `c` and the numbers. Opening the inode ends in a struct file whose `f_op` pointer is the driver's `file_operations`.

Historically a driver called `register_chrdev(major, name, fops)` and owned every minor under that major. Later, `cdev_init` / `cdev_add` plus `alloc_chrdev_region` let a driver take a range of minors. A struct class and `device_create` make the `/dev` node via `udev` or `devtmpfs`. That is still the full path for a serious driver. It is more code than a lab needs.

2.1 miscdevice: major 10, one name

`drivers/char/misc.c` implements a shared character major (`MISC_MAJOR`, 10) and a list of `miscdevice` objects. Each registered `misc` device gets a minor. `MISC_DYNAMIC_MINOR` means 'pick a free one'. The name field is what appears under `/dev/`. The `fops` you pass are wrapped by the `misc` core; your `unlocked_ioctl` is what the VFS will call after the open path has found this `misc` entry.

`vuln_lkm` fills four fields: `minor = MISC_DYNAMIC_MINOR`, `name = "vuln_lkm"`, `fops = { owner, unlocked_ioctl }`, `mode = 0666`. `mode` is an explicit lab choice so the CLI need not be root after `insmod`. A production driver would use `0600`

or a group and a udev rule. There is no `.read`, `.write`, `.llseek`, `.mmap`, `.poll`, `.compat_ioctl`. `cat /dev/vuln_lkm` therefore fails with a default error (typically `-EINVAL` from the VFS when the `fop` is `NULL`). All traffic is `ioctl`.

2.2 open, file, inode

`open("/dev/vuln_lkm", O_RDWR)` in the CLI walks the dentry, checks inode permissions, and calls the misc open path. The resulting struct file is what `unlocked_ioctl` receives as `filp`. `vuln_lkm_ioctl` ignores `filp`. It does not stash `private_data`. It does not reference-count extra objects. Two opens are two files, one `g_val`. `close` just drops the file. There is no `.release` hook.

The inode for a misc device is created by the misc class (`devtmpfs`). If you boot a tiny `initramfs` without `devtmpfs` you may need `mknod` yourself. `load.sh` `chmods 666` after `insmod` in case the `umask` or a udev rule tightened the node.

2.3 Not a PCI driver, not a platform driver

The Linux driver model (struct device, struct bus_type, probe/remove, sysfs) is how hardware is bound: PCI IDs, platform names, USB IDs, and so on. `vuln_lkm` does none of that. It does not implement `probe`. It does not own a struct device of its own beyond what `misc_register` creates. Thinking of it as 'a driver' is correct in the character-device sense and misleading in the bus-driver sense. The README states this on purpose.

2.4 /proc and /sys you will actually use

- `/proc/devices` — character majors; misc is 10
- `/proc/misc` — minor and name of each misc device
- `/sys/module/vuln_lkm/` — live module: `refcnt`, sections, holders
- `/sys/module/vuln_lkm/sections/.text` — load address for `add-symbol-file`
- `/sys/class/misc/vuln_lkm/` — the misc class device
- `dmesg / journalctl -k` — `printk` from `GET/SET/ADD/SUB/STR` and `init/exit`

After `insmod`, `cat /sys/module/vuln_lkm/sections/.text` is the address GDB needs. After `rmmod` that `sysfs` directory is gone. `lsmod` shows `vuln_lkm` and a use count; if the count is not zero, `rmmod` waits or fails because a file is still open.

2.5 Character versus block versus net

Block devices speak `bio/request` and usually appear as `/dev/sdX` or `/dev/nvmeXnY`. Network devices are struct `net_device`, not a char node (socket `ioctl` is a different path). GPIO, RTC, watchdog, and many 'control' gadgets are misc or standalone char. This lab is the last kind: a control node whose only method is `ioctl`. That is also why the CLI never calls `read(2)`.

3. ioctl(2): command words, copy_*_user, unlocked_ioctl

`ioctl` is the historical Unix escape hatch: an operation that is not `read`, `write`, or `seek`. The syscall is `ioctl(fd, cmd, arg)`. `cmd` is a 32-bit word. `arg` is an unsigned long that is either an integer or a userspace pointer. The kernel does not magically know which. The driver author and the header macros encode that contract.

3.1 _IOC packing

`include/uapi/asm-generic/ioctl.h` defines `_IOC(dir, type, nr, size)`. On Linux: `nr` in bits 0–7, `type` in 8–15, `size` in 16–29 (14 bits, max 16 KiB-1), `dir` in 30–31. Direction flags: `_IOC_NONE` 0, `_IOC_WRITE` 1 (userspace writes, kernel reads), `_IOC_READ` 2 (kernel writes, userspace reads). `_IOR` is read, `_IOW` is write, `_IOWR` is both, `_IO` is none.

`VULN_LKM_IOC_MAGIC` is `0x1B`. That is the type byte. It is not a registered global; collisions with another driver using `0x1B` would only matter if both were bound to the same `fd`, which they are not. Ordinals 1–5 are `GET`, `SET`, `ADD`, `SUB`, `STR`. Size is `sizeof(struct vuln_lkm_int) = 4` or `sizeof(struct vuln_lkm_str) = 260`.

```

1 # computed the same way <asm/ioctl.h> does
2 # dir in 31:30, size in 29:16, type in 15:8, nr in 7:0
3 GET  _IOR(0x1B,1,struct vuln_lkm_int)  = 0x80041b01
4 SET  _IOW(0x1B,2,struct vuln_lkm_int)  = 0x40041b02
5 ADD  _IOW(0x1B,3,struct vuln_lkm_int)  = 0x40041b03
6 SUB  _IOW(0x1B,4,struct vuln_lkm_int)  = 0x40041b04
7 STR  _IOW(0x1B,5,struct vuln_lkm_str)  = 0x41041b05 # size 0x104 = 260

```

The kernel switch in `vuln_lkm_ioctl` compares `cmd` to those macros as compiled into the `.ko`. The CLI prints the same numbers from its own compilation of the header. If they disagree, `ENOTTY`. Some drivers mask `_IOC_NR(cmd)` and ignore size; this one does not. That is stricter and better for a lab: a wrong size is a wrong command.

3.2 unlocked_ioctl versus old .ioctl

Until 2.6.11 the VFS called `.ioctl` under the Big Kernel Lock. `.unlocked_ioctl` is the replacement: the BKL is gone, the driver serializes itself. `.compat_ioctl` exists for 32-bit tasks on 64-bit kernels when pointers live inside the struct. `vuln_lkm`'s structs contain a `u32` and a `char` array, so the layout is the same in 32- and 64-bit userspace. There is still no `compat_ioctl`; a 32-bit CLI on a 64-bit kernel will still issue the same command words because size is 4 or 260, not `sizeof(pointer)`.

The signature is long (`*unlocked_ioctl`)(`struct file *filp`, `unsigned int cmd`, `unsigned long arg`). Returning a negative `errno` is the error path. Returning 0 is success. Positive returns are rare for this style of driver (some `ioctls` return a length).

3.3 arg is a user pointer; copy_from_user

For every command in this lab, `arg` is a pointer into the CLI's stack. The kernel must not dereference it as a kernel pointer. `copy_from_user(dst, src, n)` checks `access_ok`, then copies, and returns the number of bytes not copied. A nonzero return becomes `-EFAULT` in `copy_int_from_user` and in `vuln_lkm_str`. `copy_to_user` is the opposite, used by `GET`.

Those helpers can sleep: a page of userspace might be swapped out, the fault handler may block. That is legal only in process context (see the `IRQ` chapter). `ioctl` from a user task is process context. Calling `copy_from_user` from a hardirq handler is a bug.

After `STR`'s `copy_from_user`, `req` lives on the kernel stack. `memcpy(kbuf, req.data, req.len)` is kernel-to-kernel. It does not take a page fault on the user address. It also does not consult `kbuf`'s size. That is the overflow.

3.4 Why not read/write

You could imagine `GET` as `read(2)` of 4 bytes and `SET` as `write(2)`. `ioctl` is used instead because the lab wants a command discriminator (add versus sub versus str) and a structured argument. That is the usual reason control devices pick `ioctl`: several operations, not a stream. The cost is the packing ABI and the lack of a simple `cat/echo` path.

3.5 ENOTTY and friends

The default branch returns `-ENOTTY` ('inappropriate `ioctl` for device'). Userspace sees that as `ioctl` failed: Inappropriate `ioctl` for device. `EFAULT` is a bad user pointer. `EINTR` can happen if a signal hits a sleeping copy. The CLI does not retry. After `rmmod`, an existing `fd` may return `ENODEV` on the next `ioctl` depending on core behavior; a new open fails with `ENOENT`.

4. IRQs (Windows) and the Linux equivalents

`IRQ` is a Windows kernel concept: Interrupt Request Level. It is a per-processor number. Raising `IRQ` masks interrupts at that level and below. Code at a given `IRQ` may only call APIs legal at that level. The names you will hear from Windows driver work are `PASSIVE_LEVEL`, `APC_LEVEL`, `DISPATCH_LEVEL`, and `DIRQL` (device `IRQ`). This lab is a Linux module. It has no `KeRaiseIrql`. The mapping is still worth learning because many people coming from minifilters ask 'what `IRQ` is `ioctl`'?

4.1 Windows: the ladder

PASSIVE_LEVEL (0) is normal thread context. Paged memory is reachable. The thread can wait on dispatcher objects. Most IRP_MJ_CREATE / IRP_MJ_DEVICE_CONTROL handling for a simple DO can run here if the driver did not otherwise raise. APC_LEVEL (1) is still thread context but APCs are masked. DISPATCH_LEVEL (2) is where the Windows scheduler's DPC/dispatch sits. You must not page-fault, must not wait, must not touch paged pool. Spinlocks in the NT kernel raise to DISPATCH_LEVEL. DIRQL is device-specific, higher than dispatch, used in ISRs. The ISR typically queues a DPC that runs at DISPATCH_LEVEL.

A Windows ioctl (IRP_MJ_DEVICE_CONTROL) for a user handle is normally delivered at PASSIVE_LEVEL in the calling thread (or a worker). If a driver calls KeRaiseIrql to DISPATCH_LEVEL inside that path, copy-from-user equivalents that can fault become illegal. That is the discipline.

4.2 Linux: there is no IRQ number

Linux tracks context with preempt_count() and flags: hardirq, softirq, NMI, and 'in process context'. in_irq() is true in a hardirq handler. in_softirq() is true in a softirq/tasklet/timer callback (BH). in_interrupt() is those plus NMI. in_nmi() is NMI. If none of those, you are in process context: a task, possibly in kernel mode because of a syscall, fault, or kernel thread.

Process context may sleep (unless a spinlock is held, or preempt_disable, or you are in an atomic region). Interrupt context must not sleep. GFP_KERNEL allocations may sleep; GFP_ATOMIC must not. copy_from_user may sleep. printk is generally safe. mutex_lock may sleep; spin_lock does not sleep but disables preemption (and on RT kernels the story is more nuanced).

4.3 Rough dictionary

- PASSIVE_LEVEL ≈ process context, might sleep. ioctl(2) into vuln_lkm is this.
- APC_LEVEL ≈ no exact Linux twin; Linux does not mask APCs as NT does.
- DISPATCH_LEVEL ≈ cannot sleep, like holding a spinlock or running a softirq/DPC analogue. Tasklets and most softirqs are the closest daily analogue of DPCs.
- DIRQL ≈ hardirq handler for a device; top-half ISR. Keep it short, no copy_from_user.
- KeRaiseIrql(DISPATCH_LEVEL) ≈ spin_lock_bh / local_bh_disable / spin_lock_irq depending what you mask.
- Paged pool vs nonpaged ≈ Linux has one kernel map; 'can I fault?' is about might_sleep versus atomic, not a separate pool type with the same name.

4.4 This lab's ioctl runs at process context

When vuln_lkm_cli calls ioctl, the CPU executes the syscall entry, VFS, misc, then vuln_lkm_ioctl in the CLI's task. preempt_count is not in hardirq. copy_from_user is legal. printk is legal. A mutex would be legal (we do not take one). A spinlock would also be legal but would make copy_from_user illegal if held across the copy — we do not take one. g_val is a naked static u32; concurrent ioctls can tear it. The CLI is single-threaded in the documented exercises.

You will not see IRQ PASSIVE_LEVEL in crash dumps of this module. You will see a process name (vuln_lkm_cli) in an oops, a syscall, and a kernel stack through ksys_ioctl / vfs_ioctl / vuln_lkm_ioctl / vuln_lkm_str. That is the Linux way of saying 'this was a blocking-capable user thread in the kernel.'

4.5 What would be wrong

If someone moved memcpy of user memory into a timer callback, a tasklet, or an IRQ handler, that would be the Linux equivalent of doing user copies at DISPATCH_LEVEL or DIRQL. This source does not do that. If someone called spin_lock_irqsave and then copy_from_user, lockdep and might_sleep checks (with CONFIG_DEBUG_ATOMIC_SLEEP) would complain. The lab's bugs are wrap and kbuf size, not context bugs.

4.6 Preemption, BH, and irq flags in one paragraph

`local_irq_disable` masks `hardirqs` on this CPU. `local_bh_disable` masks `softirqs/tasklets`. `preempt_disable` masks preemption. A `spin_lock` on a non-RT kernel disables preemption. `spin_lock_bh` also disables BH. `spin_lock_irq` saves irq flags and disables IRQs. RCU read-side critical sections are another rule set. None of these appear in `vuln_lkm.c`. Read them so a later oops that says 'scheduling while atomic' means something.

4.7 Minifilter habit versus this file

A Windows minifilter pre/post callback has a documented IRQL. Some paths are `PASSIVE`, some can be `DISPATCH` (for example certain completion paths). People coming from that world look for an equivalent annotation on `unlocked_ioctl`. The Linux annotation is implicit: syscall context, may sleep, user pointer via `__user`, use `copy_to_user`. If you need to touch the device from atomic context, you design a workqueue or a threaded IRQ and do the sleeping work there. This lab never leaves process context.

5. Integer overflow and underflow

C's integers are finite. The lab uses `u32`, the kernel's typedef for a 32-bit unsigned value (`uint32_t` in userspace). The range is 0 .. 4294967295. Arithmetic on unsigned types in C is defined to wrap modulo 2^n . That is not undefined behavior. It is modular arithmetic. The bug in a driver is almost never 'unsigned wrap is UB'. The bug is 'this wrapped value is then used as a size, an index, a remaining-space check, or a value the user was told would be monotonic.'

5.1 Signed versus unsigned

Signed overflow in C is undefined behavior. Compilers may assume it never happens, fold conditions, and delete checks. Kernel code still contains signed ints (return codes, `ssize_t`). The lab avoids that trap by using `u32` for `g_val`. Unsigned wrap is defined and easy to demonstrate: `0xffffffff + 3 == 1`. Signed wrap would be the wrong lesson and a worse demo (the compiler could eat it).

Underflow of unsigned is the same modular rule: `10u - 11u == 0xffffffff`. People say 'underflow' for subtraction that crosses zero and 'overflow' for addition that crosses the max. Both are wrap. `printk` in `vuln_lkm_add` reports `wrap=yes` when the result is less than before. `vuln_lkm_sub` reports `wrap=yes` when the result is greater than before. Those detectors are for the lab log, not a security boundary.

5.2 What a check would look like

Saturation or reject: for add, if `(n > U32_MAX - g_val)` return `-EINVAL`; else `g_val += n`. For sub, if `(n > g_val)` return `-EINVAL`; else `g_val -= n`. The kernel has overflowing add helpers (`check_add_overflow` in `overflow.h`) that return a bool and a result. This module does not call them. The comment in the source says so.

A size-add check in a real driver often looks like: `new = count + extra; if (new < count) return -E2BIG; before kmalloc(new)`. The wrap made 'new' small, allocation succeeds, then a copy of 'count' bytes overflows the small buffer. That pattern is the classic integer-overflow-to-heap-overflow. This lab does not allocate. It stores the wrapped `u32` in `g_val` so you can GET it and see the number.

5.3 Widths: u8, u16, u32, u64, size_t

`size_t` is the width of an object size on the machine (64-bit on `x86_64`). Mixing a `u32` length from an `ioctl` with a `size_t` copy is a common real bug: a huge `size_t` truncated to `u32`, or a `u32` promoted and compared wrong. Here the `ioctl` value is `u32` and `g_val` is `u32` and the add is `u32`. `STR's len` is `u32` compared to 256 then passed to `memcpy` as `size_t` (the `u32` promotes). A `len` of 40 is 40. No truncation trick; the overflow is the destination size.

5.4 The lab sequences

Default `g_val` is 100. GET confirms it. SET 10; SUB 11; GET → 4294967295. SET 0xffffffff; ADD 3; GET → 1. Those two sequences are the integer chapter of the CLI help. `dmesg` will show `wrap=yes` on the ADD and SUB lines. GDB: `break vuln_lkm_add, print before, print n, stepi across g_val = g_val + n, print g_val`.

5.5 Why printk's wrap detector is not a proof of UB

`g_val < before` after an add means wrap for unsigned add of positive `n` (and `n=0` never wraps). If `n` were allowed to be interpreted as signed, the detector would be wrong. It is u32 throughout. Do not cite this `printk` as 'the kernel detected UB'. It detected modular wrap, which is exactly what the C abstract machine specified.

6. Buffer overflows in kernel C

A buffer overflow is a write (or read) past the end of an object. In this lab the object is `char kbuf[VULN_LKM_KBUF]` with `VULN_LKM_KBUF = 32`, automatic storage in `vuln_lkm_str`. The write is `memcpy(kbuf, req.data, req.len)` after `req.len` was capped only against 256. If the user sent 40 bytes, 40 bytes are written. Eight bytes (at least) land on whatever the compiler placed after `kbuf`: padding, a stack canary, saved registers, the rest of `req`, the return address.

6.1 Why the cap at 256 is not a cap at 32

The `ioctl` struct has `data[256]`. `copy_from_user` of `sizeof(req)` is 260 bytes and is fine: `dst` is struct `vuln_lkm_str req` on the stack, which is 260 bytes. The cap if (`req.len > 256`) `req.len = 256` keeps `memcpy` from reading past `req.data`. It does not keep `memcpy` from writing past `kbuf`. Two sizes exist. The code checks the wrong one for the destination. That is the whole string bug.

6.2 Stack, canaries, panic

x86_64 System V kernel ABI: local variables, then (if `CONFIG_STACKPROTECTOR`) a canary, then saved frame pointer (if any), then return address. gcc may reorder `kbuf` and `req`. A 40-byte copy into 32 bytes is a small smash. It may corrupt `req` after `kbuf`, or the canary. If the canary is hit, `__stack_chk_fail` panics. If you smash a return address and the canary is off or not covering this frame, return from `vuln_lkm_str` goes somewhere else and you oops. This PDF will not teach you to aim that write. The lesson stops at: the length used by `memcpy` was not `sizeof(kbuf)`.

A 5-byte STR ('hello') fits. `memset(kbuf,0,32)` then `memcpy` 5 bytes leaves a C string in `kbuf` that `printk` does not even print (the code logs `len` and the `kbuf` size, not the contents). A 32-byte STR fills `kbuf` exactly if there is no extra NUL write; `memcpy` of 32 is in-bounds. 33 and above are overflows. The help text uses 40 A's so it is obviously past 32.

6.3 copy_from_user is not memcpy

`copy_from_user` can fail and will not write kernel memory from a bad user pointer. `memcpy` trusts its arguments. Once the bytes are in `req`, they are kernel data. Treating `req.len` as the destination length is the same class of bug as trusting a user count after a successful copy. Hardening (`FORTIFY_SOURCE`, `__builtin_object_size`) can catch some `memcpy(dest, src, n)` when `dest`'s size is known at compile time. A lab kernel may or may not panic with a fortify message instead of a silent smash. Either way the source is wrong.

6.4 Heap versus stack (context only)

This module never `kmallocs` the string. A heap version would `kmalloc(req.len)` after a bad size check and then overflow a too-small object, or wrap the size and allocate 0. We do not implement that. If you already know HEVD-style heap labs, do not port them into this repo. The educational object here is the 32 versus 256 mismatch.

6.5 Process context still oopses

An overflow in `ioctl` is still a kernel overflow. User context does not make it a userspace bug. The oops, if any, is in kernel mode. The guest dies or the host dies depending where the `.ko` was loaded. QEMU contains that.

7. Install: headers, QEMU, GDB

Two different kernels get involved and mixing them is the usual failure. Kernel A is the one that will load the `.ko` (your WSL/Ubuntu host, or the guest you boot in QEMU). Kernel B is whatever you used to compile. `vermagic` and `Module.symvers` must match A. `linux-headers-$(uname -r)` on the machine that runs `insmod` is the right tree. A random mainline checkout is the wrong tree unless that is actually what boots.

7.1 Packages the script installs

`sudo ./scripts/install_qemu_gdb.sh` must run as root. It reads `/etc/os-release` and then:

- Debian/Ubuntu/Mint: `apt qemu-system-x86 gdb gdb-multiarch build-essential flex bison libncurses-dev libssl-dev libelf-dev linux-headers-$(uname -r)`
- Fedora/RHEL/CentOS: `dnf qemu-system-x86 gdb gcc make kernel-devel`
- Arch: `pacman qemu-system-x86 gdb base-devel`
- Anything else: the script exits and tells you to install `qemu-system-x86_64` and `gdb` yourself

`gdb-multiarch` is useful if you ever debug a non-x86 guest from an x86 host. This lab's `run_qemu.sh` is `qemu-system-x86_64`. `build-essential` / `gcc` / `make` are for the CLI and the `kbuild` wrapper. `flex`, `bison`, `ncurses`, `openssl`, `elf` are the usual 'I might compile a kernel' set; you need them for a from-source `bzImage`, not for an out-of-tree `.ko` against distro headers.

7.2 Expected installer output

```
1 $ sudo ./scripts/install_qemu_gdb.sh
2 distro: ubuntu 20.04
3 Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
4 ...
5 Setting up qemu-system-x86 ...
6 Setting up gdb ...
7
8 installed:
9 /usr/bin/qemu-system-x86_64
10 QEMU emulator version 4.2.1 (Debian 1:4.2-3ubuntu6)
11 /usr/bin/gdb
12 GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.1) 9.2
13 done.
```

Version strings vary by distro. The script prints command `-v` and the first line of `--version`. If `linux-headers` for your exact `uname -r` are missing, `apt` may fail that package and the script falls back to `qemu+gdb+build-essential` only. Then `make` module will fail until you install the matching headers.

7.3 A bzImage for QEMU

`run_qemu.sh` does not download a kernel. `KERNEL` defaults to `./bzImage` in the repo root. Build one with `CONFIG_DEBUG_INFO`, `CONFIG_GDB_SCRIPTS` if you want, and boot with `nokaslr` (the script already appends `nokaslr`). You also need a userspace: an `initramfs` that can `insmod` and run the CLI, or a disk image. `INITRD=` is optional in the script. A common lab setup is a tiny `busybox` `initramfs` plus the `9p` mount of the repo.

`9p`: `run_qemu.sh` exports the repo as `fsdev local path=$ROOT`, `mount_tag modshare`, `virtio-9p-pci`. In the guest, if the `initramfs` has `9p`: `mkdir -p /mod && mount -t 9p -o trans=virtio modshare /mod && insmod /mod/vuln_lkm.ko`. The `.ko`

must have been built against the guest kernel, not against the host WSL kernel, unless they are the same image — they are not. Cross-building an LKM for a guest means KDIR= to that kernel's build tree.

7.4 Host-only path (no QEMU)

For GET/SET/ADD/SUB you may load the module on a throwaway VM or a WSL kernel you can reboot. make; ./load.sh; ./vuln_lkm_cli get. Skip STR lengths above 32. This is enough to see integer wrap in dmesg. GDB on a live host kernel is possible (crash, /proc/kcore, kgdb) but the supported path in this repo is QEMU's -gdb stub.

8. Host build and load

The Makefile is two targets. module runs \$(MAKE) -C \$(KDIR) M=\$(CURDIR) modules with KDIR default /lib/modules/\$(uname -r)/build. That is kbuild. It produces vuln_lkm.ko, vuln_lkm.o, .mod files, and Module.symvers in the tree. user compiles vuln_lkm_cli from vuln_lkm_cli.c and the header with gcc -Wall -O2. all does both. clean runs kbuild clean and removes leftover CLI names.

```

1 obj-m += vuln_lkm.o
2 KDIR ?= /lib/modules/$(shell uname -r)/build
3
4 .PHONY: all module user clean
5
6 all: module user
7
8 module:
9     $(MAKE) -C $(KDIR) M=$(CURDIR) modules
10
11 user: vuln_lkm_cli
12
13 vuln_lkm_cli: vuln_lkm_cli.c vuln_lkm.h
14     gcc -Wall -O2 -o $@ vuln_lkm_cli.c
15
16 clean:
17     $(MAKE) -C $(KDIR) M=$(CURDIR) clean
18     rm -f vuln_lkm_cli mini_vuln_cli int_bounds_cli

```

8.1 Expected make output

```

1 $ make
2 make -C /lib/modules/5.4.0-xxx-generic/build M=/path/to/Vuln-LKM modules
3 CC [M] /path/to/Vuln-LKM/vuln_lkm.o
4 Building modules, stage 2.
5 MODPOST 1 modules
6 CC [M] /path/to/Vuln-LKM/vuln_lkm.mod.o
7 LD [M] /path/to/Vuln-LKM/vuln_lkm.ko
8 gcc -Wall -O2 -o vuln_lkm_cli vuln_lkm_cli.c

```

8.2 load.sh and unload.sh

```

1 #!/bin/sh
2 set -e
3 make module
4 sudo rmmod vuln_lkm 2>/dev/null || true
5 sudo rmmod mini_vuln 2>/dev/null || true
6 sudo rmmod int_bounds 2>/dev/null || true
7 sudo insmod ./vuln_lkm.ko
8 sudo chmod 666 /dev/vuln_lkm 2>/dev/null || true
9 echo "loaded /dev/vuln_lkm"
10 ls -l /dev/vuln_lkm

1 #!/bin/sh
2 set -e
3 sudo rmmod vuln_lkm
4 echo "unloaded vuln_lkm"

```

load.sh tries rmmod of vuln_lkm, mini_vuln, and int_bounds so leftover names from older trees do not occupy the misc name. insmod ./vuln_lkm.ko, then chmod 666 /dev/vuln_lkm if the node exists. unload.sh is rmmod vuln_lkm. After load, dmesg should contain: vuln_lkm: loaded, /dev/vuln_lkm, g_val=100.

8.3 Expected host CLI + dmesg

```

1 $ ./load.sh
2 $ dmesg | tail -n 3
3 [ 412.101234] vuln_lkm: loaded, /dev/vuln_lkm, g_val=100
4
5 $ ./vuln_lkm_cli get
6 --- GET ---
7 ioctl cmd: 0x80041b01
8 number buffer: 0 (0x00000000)
9 payload b64: AAAAAA==
10 ioctl ok
11 kernel value: 100 (0x00000064)
12 returned b64: ZAAAAA==
13
14 $ ./vuln_lkm_cli set 10
15 $ ./vuln_lkm_cli sub 11
16 $ ./vuln_lkm_cli get
17 kernel value: 4294967295 (0xffffffff)
18
19 $ dmesg | tail -n 5
20 [ 412.101234] vuln_lkm: loaded, /dev/vuln_lkm, g_val=100
21 [ 413.000001] vuln_lkm: GET 100 (0x64)
22 [ 413.100002] vuln_lkm: SET 100 -> 10
23 [ 413.200003] vuln_lkm: SUB 10 - 11 = 4294967295 (wrap=yes)
24 [ 413.300004] vuln_lkm: GET 4294967295 (0xffffffff)
25
26 $ ./unload.sh
27 [ 420.000000] vuln_lkm: unloaded

```

9. QEMU lab: expected serial output

scripts/run_qemu.sh starts qemu-system-x86_64 with 1024M RAM, -cpu max, one SMP thread, -nographic (serial is stdio), -gdb tcp::\$GDB_PORT (default 1234), -kernel \$KERNEL, append console=ttyS0 nokaslr earlyprintk=serial, and a virtio 9p share of the repo. Optional -initrd if INITRD is set. exec replaces the shell with QEMU so the GDB port stays up until you quit QEMU (Ctrl-A X in -nographic).

```

1 #!/bin/sh

```

```
2 # Boot a guest with the GDB stub on :1234 so scripts/debug_gdb.sh can attach.
3 #
4 # Required: a kernel image. Optional initramfs.
5 #   KERNEL=/path/to/bzImage INITRD=/path/to/initramfs.cpio.gz ./scripts/run_qemu.sh
6 #
7 # The repo is exported into the guest as 9p mount tag "modshare".
8 # Inside the guest (if your init mounts 9p):
9 #   mkdir -p /mod && mount -t 9p -o trans=virtio modshare /mod
10 #   insmod /mod/vuln_lkm.ko
11 #
12 # -s is shorthand for -gdb tcp::1234
13 # nokaslr keeps kernel text stable for breakpoints.
14 set -e
15
16 ROOT="$(CDPATH= cd -- "$(dirname "$0")/.." && pwd)"
17 KERNEL="${KERNEL:-$ROOT/bzImage}"
18 INITRD="${INITRD:-}"
19 GDB_PORT="${GDB_PORT:-1234}"
20 MEM="${MEM:-1024M}"
21
22 if [ ! -f "$KERNEL" ]; then
23     echo "no kernel image at $KERNEL" >&2
24     echo "set KERNEL=/path/to/bzImage (debug build, nokaslr recommended)" >&2
25     echo "see Docs/install-and-run.md" >&2
26     exit 1
27 fi
28
29 echo "QEMU kernel=$KERNEL"
30 echo "QEMU gdb stub tcp::$GDB_PORT (then run ./scripts/debug_gdb.sh)"
31 echo "QEMU 9p share $ROOT -> mount tag modshare"
32 echo "append: console=ttyS0 nokaslr"
33
34 set -- qemu-system-x86_64 \
35     -m "$MEM" \
36     -cpu max \
37     -smp 1 \
38     -nographic \
39     -gdb "tcp::$GDB_PORT" \
40     -kernel "$KERNEL" \
41     -append "console=ttyS0 nokaslr earlyprintk=serial" \
42     -fsdev "local,id=mod,path=$ROOT,security_model=none" \
43     -device "virtio-9p-pci,fsdev=mod,mount_tag=modshare"
44
45 if [ -n "$INITRD" ]; then
46     if [ ! -f "$INITRD" ]; then
47         echo "INITRD not found: $INITRD" >&2
48         exit 1
49     fi
50     set -- "$@" -initrd "$INITRD"
51     echo "QEMU initrd=$INITRD"
52 fi
53
54 exec "$@"
```

9.1 Terminal 1: launch

```

1 $ export KERNEL=/path/to/bzImage
2 $ export INITRD=/path/to/initramfs.cpio.gz # optional
3 $ ./scripts/run_qemu.sh
4 QEMU kernel=/path/to/bzImage
5 QEMU gdb stub tcp::1234 (then run ./scripts/debug_gdb.sh)
6 QEMU 9p share /path/to/Vuln-LKM -> mount tag modshare
7 append: console=ttyS0 nokaslr
8 QEMU initrd=/path/to/initramfs.cpio.gz
9
10 earlyprintk: serial
11 Linux version 6.x.x ...
12 ...
13 [ 0.000000] Kernel command line: console=ttyS0 nokaslr earlyprintk=serial
14 [ 0.000000] KASLR disabled
15 ...
16 / #

```

If KERNEL is missing the script prints no kernel image at ... and exits 1 without starting QEMU. If the image is not an x86_64 bzImage, QEMU may start and then reset. nokaslr on the command line only works if the kernel was built to honor it; a kernel without RANDOMIZE_BASE will also have a stable text, which is fine for breakpoints.

9.2 Inside the guest: load the module

```

1 / # mkdir -p /mod
2 / # mount -t 9p -o trans=virtio modshare /mod
3 / # ls /mod
4 Makefile vuln_lkm.c vuln_lkm.h vuln_lkm.ko vuln_lkm_cli ...
5 / # insmod /mod/vuln_lkm.ko
6 [ 12.401122] vuln_lkm: loaded, /dev/vuln_lkm, g_val=100
7 / # ls -l /dev/vuln_lkm
8 crw-rw-rw- 1 root root 10, 55 Aug 29 12:00 /dev/vuln_lkm
9 / # cat /sys/module/vuln_lkm/sections/.text
10 0xffffffffc0000000
11 / # /mod/vuln_lkm_cli get
12 --- GET ---
13 ioctl cmd: 0x80041b01
14 ...
15 kernel value: 100 (0x00000064)

```

The .text address is an example. Your guest will print a real address; that is the one you pass to add-symbol-file. If insmod says Invalid module format, the .ko was built for another kernel. Rebuild with KDIR pointing at this guest's build tree, or build the module on a filesystem that is the guest's own /lib/modules/\$(uname -r)/build.

If 9p is missing from the initramfs, the mount fails. Workaround: rebuild the initramfs with the .ko and CLI inside, or use a disk image that already contains them. The script will not copy files for you.

9.3 QEMU GDB stub notes

-gdb tcp::1234 waits for a debugger. QEMU still runs; -S is not passed, so the guest is not frozen at reset. You can type in the serial console while GDB is disconnected. When GDB connects (target remote), it stops the vCPU. continue resumes. If something else already bound 1234, QEMU fails to open the stub; set GDB_PORT and use the same port in a hand-run gdb (the .gdb script is hardcoded to localhost:1234; edit it if you change the port).

-nographic multiplexes the monitor onto the console. Ctrl-A C switches to the monitor, Ctrl-A X quits. Do not send Ctrl-C into QEMU if you meant it for GDB; GDB is the other terminal.

9.4 Why nokaslr

KASLR slides kernel text. Module addresses still vary per insmod even with nokaslr, which is why add-symbol-file uses /sys/module/.../text. nokaslr keeps vmlinux text stable so a vmlinux with debug info matches. For this lab you often debug only the .ko and can live without vmlinux, but breakpoints on kernel helpers (copy_from_user, printk) need guest vmlinux or kallsyms.

10. GDB remote: expected session

Terminal 2, while QEMU is running. scripts/debug_gdb.sh locates gdb, prints the four breakpoint names, and execs gdb -q -x scripts/vuln_lkm.gdb with vuln_lkm.ko as an extra argument if the file exists. The .gdb script turns pagination off, target remote localhost:1234, and break on vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str.

```

1 #!/bin/sh
2 # Attach GDB to the QEMU stub started by scripts/run_qemu.sh.
3 # Breakpoints: vuln_lkm_ioctl (dispatch), vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str.
4 set -e
5
6 ROOT="$(CDPATH= cd -- "$(dirname "$0")/.." && pwd)"
7 GDB="{GDB:-gdb}"
8 SYMBOLS="{SYMBOLS:-$ROOT/vuln_lkm.ko}"
9 GDBSCRIPT="$ROOT/scripts/vuln_lkm.gdb"
10
11 if ! command -v "$GDB" >/dev/null 2>&1; then
12     echo "gdb not found; run sudo ./scripts/install_qemu_gdb.sh" >&2
13     exit 1
14 fi
15
16 if [ ! -f "$GDBSCRIPT" ]; then
17     echo "missing $GDBSCRIPT" >&2
18     exit 1
19 fi
20
21 echo "connecting to QEMU gdb stub localhost:1234"
22 echo "breakpoints: vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str"
23 if [ -f "$SYMBOLS" ]; then
24     echo "module file: $SYMBOLS (add-symbol-file after insmod; see Docs/install-and-run.md)"
25 fi
26
27 exec "$GDB" -q -x "$GDBSCRIPT" "${SYMBOLS:+"$SYMBOLS"}"

```

```

1 # GDB commands for vuln_lkm. Sourced by scripts/debug_gdb.sh
2 set pagination off
3 set confirm off
4
5 # Guest gdb stub (run_qemu.sh uses -gdb tcp::1234)
6 target remote localhost:1234
7
8 # Dispatcher + one function per vulnerability.
9 break vuln_lkm_ioctl
10 break vuln_lkm_add
11 break vuln_lkm_sub
12 break vuln_lkm_str
13
14 printf "\nBreakpoints for vuln_lkm:\n"
15 info breakpoints
16 printf "\nIf these pending breaks never resolve, the .ko is not loaded yet.\n"
17 printf "In the guest: insmod vuln_lkm.ko\n"
18 printf "Then add-symbol-file vuln_lkm.ko <addr> where addr is:\n"

```

```
19 printf " cat /sys/module/vuln_lkm/sections/.text\n\n"
```

10.1 First attach (module not loaded yet)

```
1 $ ./scripts/debug_gdb.sh
2 connecting to QEMU gdb stub localhost:1234
3 breakpoints: vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str
4 module file: /path/to/Vuln-LKM/vuln_lkm.ko (add-symbol-file after insmod)
5
6 Remote debugging using localhost:1234
7 default_idle () at /build/linux/arch/x86/kernel/process.c:NNN
8 warning: Loadable module `vuln_lkm.ko' is not in the target
9 Function "vuln_lkm_ioctl" not defined.
10 Make breakpoint pending on future shared library load? (y or [n])
11 (confirm is off; pending is created by the script's break lines)
12
13 Breakpoints for vuln_lkm:
14 Num      Type           Disp Enb Address            What
15 1        breakpoint      keep y   <PENDING>          vuln_lkm_ioctl
16 2        breakpoint      keep y   <PENDING>          vuln_lkm_add
17 3        breakpoint      keep y   <PENDING>          vuln_lkm_sub
18 4        breakpoint      keep y   <PENDING>          vuln_lkm_str
19
20 If these pending breaks never resolve, the .ko is not loaded yet.
21 In the guest: insmod vuln_lkm.ko
22 Then add-symbol-file vuln_lkm.ko <addr> where addr is:
23 cat /sys/module/vuln_lkm/sections/.text
```

Pending breakpoints are normal before insmod. GDB cannot bind a name it has no symbols for. After insmod in the guest, you still often need add-symbol-file because a remote QEMU stub does not automatically load .ko symbols from the host path.

10.2 add-symbol-file after insmod

```
1 (gdb) continue
2 Continuing.
3
4 # (in the QEMU serial terminal)
5 / # insmod /mod/vuln_lkm.ko
6 / # cat /sys/module/vuln_lkm/sections/.text
7 0xffffffffc0002000
8
9 # (back in GDB: interrupt with Ctrl-C, or the stub stops on next break)
10 (gdb) add-symbol-file vuln_lkm.ko 0xffffffffc0002000
11 add symbol table from file "vuln_lkm.ko" at .text_addr = 0xffffffffc0002000
12 (y or n) y
13 Reading symbols from vuln_lkm.ko...
14 (gdb) info breakpoints
15 Num      Type           Disp Enb Address            What
16 1        breakpoint      keep y   0xffffffffc0002xxx in vuln_lkm_ioctl at vuln_lkm.c:63
17 2        breakpoint      keep y   0xffffffffc0002xxx in vuln_lkm_add at vuln_lkm.c:17
18 3        breakpoint      keep y   0xffffffffc0002xxx in vuln_lkm_sub at vuln_lkm.c:29
19 4        breakpoint      keep y   0xffffffffc0002xxx in vuln_lkm_str at vuln_lkm.c:41
20 (gdb) continue
```

Addresses above are examples. Use the guest's .text. If add-symbol-file complains about the path, pass an absolute host path to the same .ko you insmod'd (the 9p share is that file). If line numbers are missing, the .ko was built without debug info; rebuild with EXTRA_CFLAGS=-g or the kernel's CONFIG_DEBUG_INFO and a debug-enabled kbuild.

10.3 Useful GDB commands in this lab

- `continue / c` — run until a breakpoint or the guest is interrupted
- `step / s` — source step (needs debug info)
- `next / n` — step over a call
- `stepi / si` — one instruction; use this across `g_val = g_val + n`
- `finish` — run to the end of the current function
- `bt` — kernel backtrace: ioctl syscall frames above `vuln_lkm_*`
- `info registers / p $rdi / p $rsi` — System V: `rdi=filp`, `rsi=cmd`, `rdx=arg` on entry to `ioctl`
- `p /x cmd p g_val p n p before p req.len p &kbuf p sizeof(kbuf)`
- `x/8gx $rsp` — raw stack near `kbuf` after a too-long STR (observe, do not weaponize)
- `disconnect` then `quit` — leave QEMU running

10.4 Hitting ADD from the guest CLI

```

1 # guest
2 / # /mod/vuln_lkm_cli set 0xfffffffffe
3 / # /mod/vuln_lkm_cli add 3
4
5 # gdb
6 Breakpoint 1, vuln_lkm_ioctl (filp=0xffff..., cmd=0x40041b03, arg=0x7fff...)
7   at vuln_lkm.c:63
8 63      struct vuln_lkm_int req = {0};
9 (gdb) p /x cmd
10 $1 = 0x40041b03
11 (gdb) continue
12 Breakpoint 2, vuln_lkm_add (n=3) at vuln_lkm.c:17
13 17      u32 before = g_val;
14 (gdb) p g_val
15 $2 = 4294967294
16 (gdb) p n
17 $3 = 3
18 (gdb) n
19 22      g_val = g_val + n;
20 (gdb) n
21 23      printk(KERN_INFO "vuln_lkm: ADD %u + %u = %u (wrap=%s)\n",
22 (gdb) p g_val
23 $4 = 1
24 (gdb) continue

```

You will hit `ioctl` first, then `add`, because `add` is called from the `ADD` case. If you disable breakpoint 1, you stop only in `vuln_lkm_add`. That is often cleaner.

10.5 Hitting STR

```

1 # guest - 5 bytes, in bounds
2 / # /mod/vuln_lkm_cli str hello
3
4 # gdb
5 Breakpoint 4, vuln_lkm_str (arg=0x7fff...) at vuln_lkm.c:41
6 41      struct vuln_lkm_str req;
7 (gdb) n
8 (gdb) n
9 (gdb) p req.len
10 $10 = 5
11 (gdb) p sizeof(kbuf)
12 $11 = 32
13 (gdb) p VULN_LKM_KBUF
14 $12 = 32
15 (gdb) p req.data[0]@5
16 $13 = "hello"
17 (gdb) continue

```

For a 40-byte string, `p req.len` should be 40, `sizeof(kbuf)` remains 32. Stepping across `memcpy` may oops the guest. Set a breakpoint on the `memcpy` line, inspect, then decide whether to continue. Reboot the guest after a panic; QEMU itself should still be alive unless you quit it.

10.6 Backtrace of a successful GET

```

1 (gdb) bt
2 #0  vuln_lkm_ioctl (filp=..., cmd=0x80041b01, arg=...) at vuln_lkm.c:69
3 #1  vfs_ioctl
4 #2  ksys_ioctl
5 #3  __x64_sys_ioctl
6 #4  do_syscall_64
7 #5  entry_SYSCALL_64_after_hwframe

```

Frame names vary by kernel version. The point is: `syscall`, not `hardirq`. That is the IRQL chapter in one backtrace.

11. Step-by-step: GET / SET / ADD / SUB / STR

Do this in order the first time. Host integer wrap can be done without QEMU. STR overflow and GDB are QEMU.

11.1 GET (control)

- Build: `make`
- Load: `./load.sh` (or `insmod` in the guest)
- Run: `./vuln_lkm_cli get`
- Expect kernel value 100, `ioctl` cmd `0x80041b01`
- `dmesg`: `vuln_lkm: GET 100 (0x64)`
- GDB: break `vuln_lkm_ioctl`, `p cmd`, `p g_val`, `n` through `copy_to_user`

11.2 SET then GET

- `./vuln_lkm_cli set 10`
- `dmesg`: `SET 100 -> 10` (or `SET <old> -> 10`)
- `./vuln_lkm_cli get` → 10
- GDB: after `copy_int_from_user`, `p req.value`, `n` through the store, `p g_val`

11.3 SUB underflow

- Start from g_val=10 (previous SET)
- ./vuln_lkm_cli sub 11
- dmesg: SUB 10 - 11 = 4294967295 (wrap=yes)
- ./vuln_lkm_cli get → 4294967295 (0xffffffff)
- GDB: break vuln_lkm_sub, p before, p n, step the subtraction, p g_val

11.4 ADD overflow

- ./vuln_lkm_cli set 0xfffffffffe
- ./vuln_lkm_cli add 3
- dmesg: ADD 4294967294 + 3 = 1 (wrap=yes)
- ./vuln_lkm_cli get → 1
- GDB: break vuln_lkm_add, same inspection as SUB

11.5 STR in-bounds then out-of-bounds (QEMU)

- Guest: /mod/vuln_lkm_cli str hello
- dmesg: STR len=5 into kbuf[32]
- GDB: p req.len == 5, p sizeof(kbuf) == 32, continue; guest lives
- Guest: /mod/vuln_lkm_cli str with 40 A's (see CLI --help)
- dmesg: STR len=40 into kbuf[32]
- GDB: p req.len == 40, p &kbuf, inspect; stepping memcpy may panic
- If panic: reboot guest, insmod again, add-symbol-file again

Do not paste a 40-byte STR into a host kernel you need. The help text is explicit. The run script exists so the panic is a VM reset.

11.6 Cleanup

```
1 $ ./unload.sh
2 # or in guest: rmmod vuln_lkm
3 # QEMU: Ctrl-A X
4 # GDB: quit
```

12. Mapping CLI to kernel and breakpoints

One table for the whole lab. cmd hex is the usual x86_64 Linux packing of the macros in vuln_lkm.h.

1 CLI argv	ioctl macro	cmd hex	kernel function	GDB break
2 -----	-----	-----	-----	-----
3 get	VULN_LKM_GET	0x80041b01	vuln_lkm_ioctl	vuln_lkm_ioctl
4 set N	VULN_LKM_SET	0x40041b02	vuln_lkm_ioctl	vuln_lkm_ioctl
5 add N	VULN_LKM_ADD	0x40041b03	vuln_lkm_add	vuln_lkm_add
6 sub N	VULN_LKM_SUB	0x40041b04	vuln_lkm_sub	vuln_lkm_sub
7 str S	VULN_LKM_STR	0x41041b05	vuln_lkm_str	vuln_lkm_str

Every command hits vuln_lkm_ioctl first. ADD/SUB/STR then call the named helper. Disable the dispatcher breakpoint when you are tired of extra stops. Static functions are still visible to kgdb/gdb when the .ko has symbols; they are not exported to other modules (no EXPORT_SYMBOL).

`g_val` is a static `u32` in the module. In GDB, `p g_val` works after `add-symbol-file`. If it does not, `p *(u32*)&g_val` or info variables `g_val`. `kbuf` is on the stack of `vuln_lkm_str`; it does not exist except while that function is running.

Appendix A. Linux driver model (survey)

This appendix is background. `vuln_lkm` uses almost none of it. Read it so 'driver' in Linux is not only 'a .c file with `ioctl`'.

A.1 struct device and buses

The driver model is a tree of struct device objects hung off bus types: `pci_bus_type`, `platform_bus_type`, `usb_bus_type`, `virtio`, `i2c`, `spi`, and many more. A struct `device_driver` on that bus has a `probe` and a `remove`. Matching is bus-specific: PCI ID tables, OF compatible strings, ACPI IDs, USB VID/PID. When a device appears, the core calls `probe`. That is how a NIC driver gets its MMIO and IRQ. Character registration, if any, happens inside `probe`. `Unbind` calls `remove`.

`sysfs` (`/sys/devices/...`, `/sys/bus/pci/drivers/...`) is the userspace view of that tree. Binding can be forced by writing to `bind/unbind` files. None of this runs for `vuln_lkm`. `misc_register` creates a class device under `/sys/class/misc/` and a char node. There is no `probe`.

A.2 Character stack, in layers

VFS: struct `file`, struct `inode`, `f_op`. Character core: `cdev`, `chrdevs` hash, `major/minor`. class: `class_create`, `device_create`, `udev/devtmpfs` nodes. `misc`: a `cdev` for `major 10` plus a list. TTY: a whole other layer on top of `char` (line disciplines). ALSA, DRM, input: subsystems that expose char nodes but have their own cores. If you are writing a UART driver you talk to the `tty` layer, not raw `misc`. If you are writing a lab `ioctl` gadget, `misc` is enough.

A.3 Block and NVMe (contrast)

Block drivers register a `gendisk`, supply a `request_queue` or a `blk-mq` tag set, and handle `bio`. They do not implement `unlocked_ioctl` as the primary I/O path (there is still `ioctl` for things like `BLKGETSIZE64`). Comparing that to `vuln_lkm` is how you remember why this lab has no queue.

A.4 Net (contrast)

struct `net_device`, `ndo_start_xmit`, `NAPI poll`, sockets above. `ioctl` on a net device exists (`SIOCxxx` via `sock_ioctl`) but is not the packet path. Do not look for `sk_buff` in this module.

A.5 File operations you might add later (not in this lab)

- `.read` / `.write` — simple streams; would make `echo` and `cat` work
- `.read_iter` / `.write_iter` — `iov_iter`, preferred in modern code
- `.poll` — `select/poll/epoll`
- `.mmap` — map driver memory; easy to get wrong
- `.llseek` — if the char device has a position
- `.compat_ioctl` — 32-on-64 structs with pointers
- `.release` — drop `refcounts`, wake waiters
- `.unlocked_ioctl` — what we have

A.6 Module loader

A .ko is ELF. The loader (kernel/module.c) maps it, applies relocations against kallsyms exports, checks vermagic (kernel version, SMP, PREEMPT, compiler), checks modversions if CONFIG_MODVERSIONS, then calls the init function. MODULE_LICENSE("GPL") is required to use GPL-only exports; printk and copy_from_user are available either way, but many APIs are not. THIS_MODULE is the struct module * used as fops.owner so the module cannot be unloaded while a file is open (try_module_get on open).

rmmod calls the exit function only when the reference count is zero. If a CLI still has the fd open, unload waits or fails. livepatching and kprobes are out of scope.

A.7 Out-of-tree kbuild

obj-m += vuln_lkm.o tells kbuild to produce a module from that unit. If you split files, obj-m += vuln_lkm.o and vuln_lkm-objs := a.o b.o. EXTRA_CFLAGS += -g helps GDB. The running kernel's build directory must match. On Debian that is the linux-headers package. On a self-built kernel it is the tree you compiled, with Module.symvers generated at modules_install time.

A.8 UAPI headers

Headers under include/uapi are the contract with userspace. This lab fakes that with a single vuln_lkm.h that ifdefs __KERNEL__. Shipping the header next to the CLI is the whole ABI story. If you ever put this in-tree, the ioctl header would live under include/uapi/linux/ and the CLI would include <linux/vuln_lkm.h> from linux-libc-dev. We are not in-tree.

A.9 Device numbers, udev, devtmpfs

devtmpfs is a tmpfs the kernel populates when devices are registered. udev (systemd-udevd) may chmod, symlink, and rename. A guest with only busybox and devtmpfs will still get /dev/vuln_lkm from misc. load.sh's chmod is a belt for hosts where udev applied a stricter mode. Static mknod c 10 <minor> works if you know the minor from /proc/misc.

A.10 Concurrency primitives (not used here)

spinlock_t, mutex, rwsem, RCU, atomic_t, seqlock, wait_queue_head_t. g_val would normally be atomic_t or sit under a mutex. ioctl is re-entrant across CPUs. The lab's single-threaded CLI hides that. If you add a mutex, do not hold it across copy_from_user unless you accept that a user can stall the lock by paging; copy to a local first, then lock, then commit. That pattern is the standard ioctl sketch.

A.11 Memory: kmalloc, vmalloc, pages

kmalloc is the slab/kmalloc caches, physically contiguous, limited size. vmalloc is virtually contiguous. alloc_pages is the page allocator. GFP_KERNEL vs GFP_ATOMIC is sleep vs no-sleep. This module has none of them. STR uses stack. Stack in kernel is small (often 8 KiB or 16 KiB per thread). A struct of 260 bytes plus kbuf of 32 is acceptable. A struct of many kilobytes on stack is a stack overflow of its own, distinct from memcpy past kbuf.

A.12 printk and log levels

KERN_INFO is what the module uses. dmesg --level=info will show it. Rate limiting is not applied here. Printing user-controlled strings from STR would be a log-injection nuisance; the module prints len and the kbuf size, not the bytes. Good. Do not add printk("%s", kbuf) as a 'feature'.

Appendix B. Kernel map for this lab

A walk from the syscall instruction to memcpy and back, with the files that implement each step on a typical x86_64 Linux. Line numbers shift; names are stable enough to grep.

B.1 User to kernel

vuln_lkm_cli calls the libc wrapper ioctl(), which on x86_64 is syscall SYS_ioctl (16). The CPU hits the SYSCALL instruction. The kernel's entry_SYSCALL_64 path saves user registers, swaps GS, and lands in do_syscall_64. The syscall table dispatches to __x64_sys_ioctl, which calls ksys_ioctl.

B.2 VFS

ksys_ioctl takes the fd, fget's a struct file, and calls vfs_ioctl. vfs_ioctl handles a few common commands then calls file->f_op->unlocked_ioctl. For a misc device, the fops you registered are what run — after misc's thin wrappers, your unlocked_ioctl is vuln_lkm_ioctl.

B.3 copy_from_user on x86_64

copy_from_user is an inline or a function that eventually uses raw_copy_from_user / copy_user_generic with STAC/CLAC around user access (SMAP). A fault during the copy is handled by the exception table: the instruction's fixup returns a leftover byte count instead of oopsing. That is why a bad pointer is -EFAULT, not a panic. memcpy has no exception table for user addresses; after the copy, you are in kernel addresses only.

B.4 printk

printk walks vprintk_emit, logbuf, and optionally consoles. earlyprintk=serial in the QEMU append line makes very early boot messages go to the UART. After consoles are registered, KERN_INFO from the module appears on the serial console and in the ring buffer. dmesg reads the ring. In -nographic QEMU you see printk mixed with the shell.

B.5 misc_register

misc_register allocates a minor, creates the cdev association, and device_create's the node. misc_deregister reverses it. If registration fails, module_init returns the errno and the loader unloads the module; you never see a half-registered device from this init function because we return immediately on failure.

B.6 Stack of vuln_lkm_str

On entry: return address to vuln_lkm_ioctl, saved rbp if any, then locals. Compiler order of req and kbuf is not promised by C. GDB p &req and p &kbuf tell you the layout on your build. The overflow write starts at kbuf and grows toward higher addresses on x86 (the stack grows down, so higher addresses are toward the saved frame of the caller if kbuf is below them). Inspect; do not write a payload.

B.7 QEMU, KVM, gdbstub

qemu-system-x86_64 can use TCG (emulation) or KVM (hardware virt). The gdbstub presents a remote protocol on TCP. GDB's target remote talks that protocol: stop, step, read memory, write breakpoints (usually software int3 in guest RAM, or hardware debug registers). -gdb tcp::1234 is the same as -s but with an explicit port. -S would pause at startup until GDB continues; this lab does not pass -S so the guest boots while you are still starting GDB.

kgdb is different: it is a debugger stub inside the Linux kernel (kgdboc on a serial port). This lab uses QEMU's stub, which debugs the guest from the host without kgdb in the guest kernel. You can use both; you do not need both.

B.8 Symbols: vmlinux versus .ko

vmlinux (uncompressed) carries kernel symbols. distro kernels often strip them into vmlinux-*.debug or a dwarf package. The .ko carries module symbols. add-symbol-file relocates those symbols to the load address. Without that, p g_val does not work and breakpoints stay pending. File-backed 9p means the .ko GDB reads on the host is the same bytes the guest insmod'd, which is what you want.

B.9 CONFIG options that change the lab

- CONFIG_MISC_FILESYSTEMS / 9p / VIRTIO — needed to mount modshare
- CONFIG_MODULES — without it you cannot insmod
- CONFIG_KALLSYMS — names in oopses
- CONFIG_DEBUG_INFO — source-level GDB
- CONFIG_STACKPROTECTOR — canary panics on STR overflow
- CONFIG_FORTIFY_SOURCE — memcpy may panic with a fortify report
- CONFIG_RANDOMIZE_BASE — KASLR; we pass nokaslr
- CONFIG_DEBUG_ATOMIC_SLEEP — would catch copy_from_user in atomic context; we are not atomic
- CONFIG_MODVERSIONS — extra vermagic-like CRC checks on exports

B.10 Oops reading (if STR panics)

An oops prints RIP, a register dump, a call trace, and maybe a tainted line (O for out-of-tree). RIP inside memcpy or vuln_lkm_str is expected for a smash. RIP in __stack_chk_fail is a canary. 'invalid opcode' at a smashed return can be a jump to non-code. Reboot the guest. Copy the oops text into your notes. Do not treat it as a recipe.

Appendix C. Full source and scripts

The committed tree, reproduced so this PDF stands alone next to the repo. If the repo and this listing disagree, the repo is canonical.

C.1 vuln_lkm.h

```
1 #ifndef VULN_LKM_H
2 #define VULN_LKM_H
3
4 #ifdef __KERNEL__
5 #include <linux/ioctl.h>
6 #include <linux/types.h>
7 #else
8 #include <sys/ioctl.h>
9 #include <stdint.h>
10 typedef uint32_t __u32;
11 #endif
12
13 #define VULN_LKM_NAME "vuln_lkm"
14 #define VULN_LKM_IOC_MAGIC 0x1B
15 #define VULN_LKM_STR_MAX 256
16 #define VULN_LKM_KBUF 32
17
18 struct vuln_lkm_int {
19     __u32 value;
20 };
21
22 struct vuln_lkm_str {
23     __u32 len;
24     char data[VULN_LKM_STR_MAX];
25 };
26
27 #define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
28 #define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
29 #define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
30 #define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
31 #define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)
32
33 #endif
```

C.2 vuln_lkm.c

```

1 #include <linux/module.h>
2 #include <linux/miscdevice.h>
3 #include <linux/uaccess.h>
4
5 #include "vuln_lkm.h"
6
7 static u32 g_val = 100;
8
9 static int copy_int_from_user(unsigned long arg, struct vuln_lkm_int *out)
10 {
11     if (copy_from_user(out, (void __user *)arg, sizeof(*out)))
12         return -EFAULT;
13     return 0;
14 }
15
16 /* GDB: break vuln_lkm_add */
17 static int vuln_lkm_add(u32 n)
18 {
19     u32 before = g_val;
20
21     /* No check against U32_MAX - n. The add wraps. */
22     g_val = g_val + n;
23     printk(KERN_INFO "vuln_lkm: ADD %u + %u = %u (wrap=%s)\n",
24            before, n, g_val, (g_val < before) ? "yes" : "no");
25     return 0;
26 }
27
28 /* GDB: break vuln_lkm_sub */
29 static int vuln_lkm_sub(u32 n)
30 {
31     u32 before = g_val;
32
33     /* No check against n > g_val. The sub wraps. */
34     g_val = g_val - n;
35     printk(KERN_INFO "vuln_lkm: SUB %u - %u = %u (wrap=%s)\n",
36            before, n, g_val, (g_val > before) ? "yes" : "no");
37     return 0;
38 }
39
40 /* GDB: break vuln_lkm_str */
41 static int vuln_lkm_str(unsigned long arg)
42 {
43     struct vuln_lkm_str req;
44     char kbuf[VULN_LKM_KBUF];
45
46     if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
47         return -EFAULT;
48
49     if (req.len > VULN_LKM_STR_MAX)
50         req.len = VULN_LKM_STR_MAX;
51
52     memset(kbuf, 0, sizeof(kbuf));
53     printk(KERN_INFO "vuln_lkm: STR len=%u into kbuf[%u]\n",
54            req.len, VULN_LKM_KBUF);
55     /*
56      * kbuf is 32 bytes. req.len is not compared to VULN_LKM_KBUF.
57      * memcpy uses the user-supplied length as-is.
58      */

```

```
59     memcpy(kbuf, req.data, req.len);
60     return 0;
61 }
62
63 static long vuln_lkm_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
64 {
65     struct vuln_lkm_int req = {0};
66
67     switch (cmd) {
68     case VULN_LKM_GET:
69         req.value = g_val;
70         if (copy_to_user((void __user *)arg, &req, sizeof(req)))
71             return -EFAULT;
72         printk(KERN_INFO "vuln_lkm: GET %u (0x%x)\n", g_val, g_val);
73         return 0;
74
75     case VULN_LKM_SET:
76         if (copy_int_from_user(arg, &req))
77             return -EFAULT;
78         printk(KERN_INFO "vuln_lkm: SET %u -> %u\n", g_val, req.value);
79         g_val = req.value;
80         return 0;
81
82     case VULN_LKM_ADD:
83         if (copy_int_from_user(arg, &req))
84             return -EFAULT;
85         return vuln_lkm_add(req.value);
86
87     case VULN_LKM_SUB:
88         if (copy_int_from_user(arg, &req))
89             return -EFAULT;
90         return vuln_lkm_sub(req.value);
91
92     case VULN_LKM_STR:
93         return vuln_lkm_str(arg);
94
95     default:
96         return -ENOTTY;
97     }
98 }
99
100 static const struct file_operations vuln_lkm_fops = {
101     .owner = THIS_MODULE,
102     .unlocked_ioctl = vuln_lkm_ioctl,
103 };
104
105 static struct miscdevice vuln_lkm_dev = {
106     .minor = MISC_DYNAMIC_MINOR,
107     .name = VULN_LKM_NAME,
108     .fops = &vuln_lkm_fops,
109     .mode = 0666,
110 };
111
112 static int __init vuln_lkm_init(void)
113 {
114     int ret = misc_register(&vuln_lkm_dev);
115
116     if (ret)
117         return ret;
118     printk(KERN_INFO "vuln_lkm: loaded, /dev/%s, g_val=%u\n",
```

```
119         VULN_LKM_NAME, g_val);
120     return 0;
121 }
122
123 static void __exit vuln_lkm_exit(void)
124 {
125     misc_deregister(&vuln_lkm_dev);
126     printk(KERN_INFO "vuln_lkm: unloaded\n");
127 }
128
129 module_init(vuln_lkm_init);
130 module_exit(vuln_lkm_exit);
131 MODULE_LICENSE("GPL");
132 MODULE_AUTHOR("Damian");
133 MODULE_DESCRIPTION("Educational LKM: integer wrap and string buffer overflow");
```

C.3 vuln_lkm_cli.c

```

1 #include <errno.h>
2 #include <fcntl.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>
8
9 #include "vuln_lkm.h"
10
11 #define DEV_PATH "/dev/" VULN_LKM_NAME
12
13 static const char b64tab[] =
14     "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
15
16 static void print_help(const char *argv0)
17 {
18     printf(
19 "vuln_lkm_cli - userspace tester for the vuln_lkm educational LKM\n"
20 "\n"
21 "Usage:\n"
22 " %s interactive menu\n"
23 " %s -h|--help this help\n"
24 " %s get VULN_LKM_GET\n"
25 " %s set <n> VULN_LKM_SET\n"
26 " %s add <n> VULN_LKM_ADD (u32 wrap)\n"
27 " %s sub <n> VULN_LKM_SUB (u32 wrap)\n"
28 " %s str <string> VULN_LKM_STR (copy into 32-byte kbuf)\n"
29 "\n"
30 "<n> is decimal or 0xhex.\n"
31 "Every ioctl prints cmd hex, the number or string, and base64 of the payload.\n"
32 "Device: %s\n"
33 "\n"
34 "Examples:\n"
35 " %s get\n"
36 " %s set 10\n"
37 " %s sub 11 # 10-11 wraps to 4294967295\n"
38 " %s set 0xfffffffffe\n"
39 " %s add 3 # wraps to 1\n"
40 " %s str hello # fits in 32-byte kbuf\n"
41 " %s str AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA # 40 bytes > 32\n"
42 "\n"
43 "Interactive keys: 1 GET 2 SET 3 ADD 4 SUB 5 STR q quit\n",
44     argv0, argv0, argv0, argv0, argv0, argv0, argv0,
45     DEV_PATH,
46     argv0, argv0, argv0, argv0, argv0, argv0, argv0);
47 }
48
49 static void print_b64(const void *data, size_t n)
50 {
51     const unsigned char *p = data;
52     size_t i;
53
54     for (i = 0; i < n; i += 3) {
55         unsigned a = p[i];
56         unsigned b = (i + 1 < n) ? p[i + 1] : 0;
57         unsigned c = (i + 2 < n) ? p[i + 2] : 0;
58         unsigned triple = (a << 16) | (b << 8) | c;

```

```

59
60     putchar(b64tab[(triple >> 18) & 63]);
61     putchar(b64tab[(triple >> 12) & 63]);
62     putchar((i + 1 < n) ? b64tab[(triple >> 6) & 63] : '=');
63     putchar((i + 2 < n) ? b64tab[triple & 63] : '=');
64 }
65 }
66
67 static void show_int(const char *op, unsigned int cmd,
68                     const struct vuln_lkm_int *arg, int sending)
69 {
70     printf("\n--- %s ---\n", op);
71     printf("ioctl cmd: 0x%x\n", cmd);
72     printf("number %s: %u (0x%08x)\n",
73           sending ? "sent" : "buffer", arg->value, arg->value);
74     printf("payload b64: ");
75     print_b64(arg, sizeof(*arg));
76     printf("\n");
77 }
78
79 static void show_str(unsigned int cmd, const struct vuln_lkm_str *arg)
80 {
81     printf("\n--- STR ---\n");
82     printf("ioctl cmd: 0x%x\n", cmd);
83     printf("string sent: \"%s\"\n", arg->data);
84     printf("len sent: %u\n", arg->len);
85     printf("payload b64: ");
86     print_b64(arg, sizeof(*arg));
87     printf("\n");
88 }
89
90 static int do_int(int fd, const char *op, unsigned int cmd,
91                  struct vuln_lkm_int *arg, int sending)
92 {
93     show_int(op, cmd, arg, sending);
94     if (ioctl(fd, cmd, arg) < 0) {
95         printf("ioctl failed: %s\n", strerror(errno));
96         return -1;
97     }
98     printf("ioctl ok\n");
99     return 0;
100 }
101
102 static int parse_u32(const char *s, uint32_t *out)
103 {
104     unsigned long v;
105     char *end;
106
107     errno = 0;
108     v = strtoul(s, &end, 0);
109     if (end == s || *end != '\0' || errno)
110         return -1;
111     *out = (uint32_t)v;
112     return 0;
113 }
114
115 static int read_u32(const char *prompt, uint32_t *out)
116 {
117     char buf[64];
118

```

```
119     printf("%s", prompt);
120     fflush(stdout);
121     if (!fgets(buf, sizeof(buf), stdin))
122         return -1;
123     buf[strcspn(buf, "\n")] = '\0';
124     return parse_u32(buf, out);
125 }
126
127 static int open_dev(void)
128 {
129     int fd = open(DEV_PATH, O_RDWR);
130
131     if (fd < 0)
132         fprintf(stderr, "open %s: %s\n", DEV_PATH, strerror(errno));
133     return fd;
134 }
135
136 static int cmd_get(int fd)
137 {
138     struct vuln_lkm_int arg = {0};
139
140     if (do_int(fd, "GET", VULN_LKM_GET, &arg, 0) == 0) {
141         printf("kernel value: %u (0x%08x)\n", arg.value, arg.value);
142         printf("returned b64: ");
143         print_b64(&arg, sizeof(arg));
144         printf("\n");
145     }
146     return 0;
147 }
148
149 static int cmd_set(int fd, uint32_t v)
150 {
151     struct vuln_lkm_int arg = { .value = v };
152
153     return do_int(fd, "SET", VULN_LKM_SET, &arg, 1);
154 }
155
156 static int cmd_add(int fd, uint32_t v)
157 {
158     struct vuln_lkm_int arg = { .value = v };
159
160     return do_int(fd, "ADD", VULN_LKM_ADD, &arg, 1);
161 }
162
163 static int cmd_sub(int fd, uint32_t v)
164 {
165     struct vuln_lkm_int arg = { .value = v };
166
167     return do_int(fd, "SUB", VULN_LKM_SUB, &arg, 1);
168 }
169
170 static int cmd_str(int fd, const char *s)
171 {
172     struct vuln_lkm_str arg;
173     size_t n = strlen(s);
174
175     memset(&arg, 0, sizeof(arg));
176     if (n > VULN_LKM_STR_MAX)
177         n = VULN_LKM_STR_MAX;
178     arg.len = (uint32_t)n;
```

```
179     memcpy(arg.data, s, n);
180     show_str(VULN_LKM_STR, &arg);
181     if (ioctl(fd, VULN_LKM_STR, &arg) < 0) {
182         printf("ioctl failed: %s\n", strerror(errno));
183         return -1;
184     }
185     printf("ioctl ok\n");
186     return 0;
187 }
188
189 static int interactive(int fd)
190 {
191     char line[32];
192     uint32_t v;
193     char buf[VULN_LKM_STR_MAX];
194
195     while (1) {
196         puts("\nvuln_lkm CLI");
197         puts(" 1) GET integer");
198         puts(" 2) SET integer");
199         puts(" 3) ADD (u32 overflow wrap)");
200         puts(" 4) SUB (u32 underflow wrap)");
201         puts(" 5) STR (copy into 32-byte kernel buffer)");
202         puts(" h) help");
203         puts(" q) quit");
204         printf("> ");
205         fflush(stdout);
206         if (!fgets(line, sizeof(line), stdin))
207             break;
208         switch (line[0]) {
209             case '1':
210                 cmd_get(fd);
211                 break;
212             case '2':
213                 if (read_u32("value to SET (dec or 0xhex): ", &v))
214                     puts("bad number");
215                 else
216                     cmd_set(fd, v);
217                 break;
218             case '3':
219                 if (read_u32("value to ADD (dec or 0xhex): ", &v))
220                     puts("bad number");
221                 else
222                     cmd_add(fd, v);
223                 break;
224             case '4':
225                 if (read_u32("value to SUB (dec or 0xhex): ", &v))
226                     puts("bad number");
227                 else
228                     cmd_sub(fd, v);
229                 break;
230             case '5': {
231                 size_t n;
232
233                 printf("string to send: ");
234                 fflush(stdout);
235                 if (!fgets(buf, sizeof(buf), stdin))
236                     break;
237                 n = strlen(buf);
238                 if (n && buf[n - 1] == '\n')
```

```

239         buf[--n] = '\0';
240         cmd_str(fd, buf);
241         break;
242     }
243     case 'h':
244     case 'H':
245         print_help("vuln_lkm_cli");
246         break;
247     case 'q':
248     case 'Q':
249         return 0;
250     default:
251         puts("unknown option (h for help)");
252         break;
253     }
254 }
255 return 0;
256 }
257
258 int main(int argc, char **argv)
259 {
260     int fd, rc = 0;
261     uint32_t v;
262
263     if (argc >= 2 && (!strcmp(argv[1], "-h") || !strcmp(argv[1], "--help"))) {
264         print_help(argv[0]);
265         return 0;
266     }
267
268     fd = open_dev();
269     if (fd < 0)
270         return 1;
271
272     if (argc == 1) {
273         rc = interactive(fd);
274     } else if (!strcmp(argv[1], "get") && argc == 2) {
275         rc = cmd_get(fd);
276     } else if (!strcmp(argv[1], "set") && argc == 3 && !parse_u32(argv[2], &v)) {
277         rc = cmd_set(fd, v);
278     } else if (!strcmp(argv[1], "add") && argc == 3 && !parse_u32(argv[2], &v)) {
279         rc = cmd_add(fd, v);
280     } else if (!strcmp(argv[1], "sub") && argc == 3 && !parse_u32(argv[2], &v)) {
281         rc = cmd_sub(fd, v);
282     } else if (!strcmp(argv[1], "str") && argc == 3) {
283         rc = cmd_str(fd, argv[2]);
284     } else {
285         print_help(argv[0]);
286         rc = 1;
287     }
288
289     close(fd);
290     return rc < 0 ? 1 : rc;
291 }

```

C.4 Makefile

```
1 obj-m += vuln_lkm.o
2 KDIR ?= /lib/modules/$(shell uname -r)/build
3
4 .PHONY: all module user clean
5
6 all: module user
7
8 module:
9     $(MAKE) -C $(KDIR) M=$(CURDIR) modules
10
11 user: vuln_lkm_cli
12
13 vuln_lkm_cli: vuln_lkm_cli.c vuln_lkm.h
14     gcc -Wall -O2 -o $@ vuln_lkm_cli.c
15
16 clean:
17     $(MAKE) -C $(KDIR) M=$(CURDIR) clean
18     rm -f vuln_lkm_cli mini_vuln_cli int_bounds_cli
```

C.5 load.sh

```
1 #!/bin/sh
2 set -e
3 make module
4 sudo rmmod vuln_lkm 2>/dev/null || true
5 sudo rmmod mini_vuln 2>/dev/null || true
6 sudo rmmod int_bounds 2>/dev/null || true
7 sudo insmod ./vuln_lkm.ko
8 sudo chmod 666 /dev/vuln_lkm 2>/dev/null || true
9 echo "loaded /dev/vuln_lkm"
10 ls -l /dev/vuln_lkm
```

C.6 unload.sh

```
1 #!/bin/sh
2 set -e
3 sudo rmmod vuln_lkm
4 echo "unloaded vuln_lkm"
```

C.7 scripts/install_qemu_gdb.sh

```
1 #!/bin/sh
2 # Install QEMU (x86_64 system emulator) and GDB for debugging vuln_lkm.
3 set -e
4
5 if [ "$(id -u)" -ne 0 ]; then
6     echo "re-run with sudo: sudo $0" >&2
7     exit 1
8 fi
9
10 . /etc/os-release
11 echo "distro: ${ID:-unknown} ${VERSION_ID:-}"
12
13 case "${ID}" in
14     debian|ubuntu|linuxmint)
15         export DEBIAN_FRONTEND=noninteractive
16         apt-get update
17         apt-get install -y --no-install-recommends \
18             qemu-system-x86 \
19             gdb \
20             gdb-multiarch \
21             build-essential \
22             flex bison libncurses-dev libssl-dev libelf-dev \
23             linux-headers-$(uname -r) || apt-get install -y qemu-system-x86 gdb build-essential
24         ;;
25     fedora|rhel|centos)
26         dnf install -y qemu-system-x86 gdb gcc make kernel-devel || yum install -y qemu-system-x86 gdb
27         gcc make
28         ;;
29     arch)
30         pacman -Sy --noconfirm qemu-system-x86 gdb base-devel
31         ;;
32     *)
33         echo "unknown distro; install qemu-system-x86_64 and gdb yourself" >&2
34         exit 1
35         ;;
36 esac
37
38 echo
39 echo "installed:"
40 command -v qemu-system-x86_64
41 qemu-system-x86_64 --version | head -n1
42 command -v gdb
43 gdb --version | head -n1
44 echo "done."
```

C.8 scripts/run_qemu.sh

```

1 #!/bin/sh
2 # Boot a guest with the GDB stub on :1234 so scripts/debug_gdb.sh can attach.
3 #
4 # Required: a kernel image. Optional initramfs.
5 #   KERNEL=/path/to/bzImage INITRD=/path/to/initramfs.cpio.gz ./scripts/run_qemu.sh
6 #
7 # The repo is exported into the guest as 9p mount tag "modshare".
8 # Inside the guest (if your init mounts 9p):
9 #   mkdir -p /mod && mount -t 9p -o trans=virtio modshare /mod
10 #   insmod /mod/vuln_lkm.ko
11 #
12 # -s is shorthand for -gdb tcp::1234
13 # nokaslr keeps kernel text stable for breakpoints.
14 set -e
15
16 ROOT="$(CDPATH= cd -- "$(dirname "$0")/.." && pwd)"
17 KERNEL="${KERNEL:-$ROOT/bzImage}"
18 INITRD="${INITRD:-}"
19 GDB_PORT="${GDB_PORT:-1234}"
20 MEM="${MEM:-1024M}"
21
22 if [ ! -f "$KERNEL" ]; then
23     echo "no kernel image at $KERNEL" >&2
24     echo "set KERNEL=/path/to/bzImage (debug build, nokaslr recommended)" >&2
25     echo "see Docs/install-and-run.md" >&2
26     exit 1
27 fi
28
29 echo "QEMU kernel=$KERNEL"
30 echo "QEMU gdb stub tcp::$GDB_PORT (then run ./scripts/debug_gdb.sh)"
31 echo "QEMU 9p share $ROOT -> mount tag modshare"
32 echo "append: console=ttyS0 nokaslr"
33
34 set -- qemu-system-x86_64 \
35     -m "$MEM" \
36     -cpu max \
37     -smp 1 \
38     -nographic \
39     -gdb "tcp::$GDB_PORT" \
40     -kernel "$KERNEL" \
41     -append "console=ttyS0 nokaslr earlyprintk=serial" \
42     -fsdev "local,id=mod,path=$ROOT,security_model=none" \
43     -device "virtio-9p-pci,fsdev=mod,mount_tag=modshare"
44
45 if [ -n "$INITRD" ]; then
46     if [ ! -f "$INITRD" ]; then
47         echo "INITRD not found: $INITRD" >&2
48         exit 1
49     fi
50     set -- "$@" -initrd "$INITRD"
51     echo "QEMU initrd=$INITRD"
52 fi
53
54 exec "$@"

```

C.9 scripts/debug_gdb.sh

```

1 #!/bin/sh
2 # Attach GDB to the QEMU stub started by scripts/run_qemu.sh.
3 # Breakpoints: vuln_lkm_ioctl (dispatch), vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str.
4 set -e
5
6 ROOT="$(CDPATH= cd -- "$(dirname "$0")/.." && pwd)"
7 GDB="${GDB:-gdb}"
8 SYMBOLS="${SYMBOLS:-$ROOT/vuln_lkm.ko}"
9 GDBSCRIPT="$ROOT/scripts/vuln_lkm.gdb"
10
11 if ! command -v "$GDB" >/dev/null 2>&1; then
12     echo "gdb not found; run sudo ./scripts/install_qemu_gdb.sh" >&2
13     exit 1
14 fi
15
16 if [ ! -f "$GDBSCRIPT" ]; then
17     echo "missing $GDBSCRIPT" >&2
18     exit 1
19 fi
20
21 echo "connecting to QEMU gdb stub localhost:1234"
22 echo "breakpoints: vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str"
23 if [ -f "$SYMBOLS" ]; then
24     echo "module file: $SYMBOLS (add-symbol-file after insmod; see Docs/install-and-run.md)"
25 fi
26
27 exec "$GDB" -q -x "$GDBSCRIPT" "${SYMBOLS:+"$SYMBOLS"}"

```

C.10 scripts/vuln_lkm.gdb

```

1 # GDB commands for vuln_lkm. Sourced by scripts/debug_gdb.sh
2 set pagination off
3 set confirm off
4
5 # Guest gdb stub (run_qemu.sh uses -gdb tcp::1234)
6 target remote localhost:1234
7
8 # Dispatcher + one function per vulnerability.
9 break vuln_lkm_ioctl
10 break vuln_lkm_add
11 break vuln_lkm_sub
12 break vuln_lkm_str
13
14 printf "\nBreakpoints for vuln_lkm:\n"
15 info breakpoints
16 printf "\nIf these pending breaks never resolve, the .ko is not loaded yet.\n"
17 printf "In the guest: insmod vuln_lkm.ko\n"
18 printf "Then add-symbol-file vuln_lkm.ko <addr> where addr is:\n"
19 printf " cat /sys/module/vuln_lkm/sections/.text\n\n"

```

Appendix D. Glossary

ABI — application binary interface; here, ioctl numbers and struct layouts in vuln_lkm.h.

arg — third argument to ioctl; in this lab, a userspace pointer to vuln_lkm_int or vuln_lkm_str.

bzImage — compressed packed kernel image QEMU loads with -kernel.

cdev — struct cdev, the character-device object; misc wraps one shared cdev.

copy_from_user / copy_to_user — kernel helpers that copy to/from user addresses and return leftover bytes on fault.

DIRQL — Windows device IRQL; ISR level. Linux analogue: hardirq handler. Not used by this ioctl path.

DISPATCH_LEVEL — Windows IRQL where DPCs run and paging is illegal. Linux analogue: atomic/softirq-ish context, no sleep.

dmesg — userspace reader of the kernel log ring; printk from the module shows up here.

ENOTTY — errno for unknown ioctl command; default branch of vuln_lkm_ioctl.

file_operations — table of VFS methods; this module sets owner and unlocked_ioctl.

g_val — static u32 lab state, starts at 100, mutated by SET/ADD/SUB.

GDB stub — QEMU's remote debug server (-gdb tcp::1234).

insmod / rmmod — load and unload a .ko (also: modprobe).

ioctl — syscall that dispatches on a packed 32-bit command word.

IRQL — Windows interrupt-request level. Linux does not use this name; see process context, in_irq, in_softirq.

kbuf — char[32] stack buffer in vuln_lkm_str; destination of memcpy.

KASLR — kernel address-space layout randomization; disabled in the guest via nokaslr.

kbuild — the kernel's Make system; make -C \$KDIR M=\$PWD modules.

LKM — loadable kernel module, a .ko inserted at runtime, not linked into vmlinux.

magic — the type byte in an ioctl command; this lab uses 0x1B.

memcpy — unbounded C copy; here keyed off req.len, not sizeof(kbuf).

minor / major — device-number pair; misc uses major 10 and a dynamic minor.

miscdevice — helpers for a simple char device on the shared misc major.

module_init / module_exit — constructors called at insmod/rmmod.

nokaslr — kernel command-line flag to keep kernel text unslid.

oops / panic — kernel bug reports; STR overflow can cause them.

PASSIVE_LEVEL — Windows IRQL for normal thread context; analogue of Linux process context. This ioctl runs there.

pending breakpoint — GDB break on a name not yet in the symbol table (module not loaded).

process context — Linux: running as a task, may sleep, not in hardirq/softirq. ioctl is this.

QEMU — emulator/VMM used here to contain panics and to host a GDB stub.

req.len — user-supplied length in struct vuln_lkm_str; capped at 256, not 32.

SMAP — supervisor mode access prevention; copy_*_user temporarily allows user access.

unlocked_ioctl — file_operations method, ioctl without the old BKL.

u32 wrap — unsigned 32-bit modular arithmetic; ADD/SUB lab bugs.

UAPI — user API headers; this lab's shared header plays that role.

vermagic — string in a .ko that must match the running kernel.

VFS — virtual file system; fd to file to f_op.

9p / modshare — virtio-9p export of the repo into the guest.

D.1 ioctl command words (repeat)

```
1 GET 0x80041b01 SET 0x40041b02 ADD 0x40041b03
2 SUB 0x40041b04 STR 0x41041b05
```

Appendix E. Failure table and lab rules

1 symptom	likely cause
2 -----	-----
3 make: No such file or directory	linux-headers-\$(uname -r) missing, KDIR wrong
4 Invalid module format	.ko built for another kernel (vermagic)
5 open /dev/vuln_lkm: No such	module not loaded, misc_register failed
6 open: Permission denied	node not 0666 and you are not root
7 ioctl: Inappropriate ioctl	header skew; cmd word != module
8 QEMU exits at once	KERNEL path missing or not a bzImage
9 GDB: connection refused	QEMU not running or port != 1234
10 breakpoints PENDING forever	no insmod, or no add-symbol-file
11 p g_val unknown	symbols not loaded for the .ko
12 guest oops on str	len > 32; reboot guest
13 9p mount fails	guest kernel/initramfs lacks 9p/virtio
14 CLI prints help and exits 1	argv parse failed (need set 10 not set)

E.1 Lab rules

- Educational module. Do not ship it. Do not load it on production.
- Integer wrap demos are fine on a throwaway host kernel. Long STR is not.
- No exploit payloads, no SMEP/KASLR bypass, no privesc writeups in this tree.
- GDB is for watching g_val, req.len, and kbuf, not for developing an exploit.
- If the guest panics, reset the guest. If the host panics, that is why QEMU exists.
- The three PDFs: vuln_lkm.pdf (driver), vuln_lkm_cli.pdf (CLI), this file (install/debug/theory).

E.2 Suggested reading next to the kernel tree

Documentation/driver-api/driver-model/ — devices and buses. Documentation/filesystems/vfs.rst — file_operations. include/uapi/asm-generic/ioctl.h — packing. include/linux/uaccess.h — copy_*_user. include/linux/miscdevice.h — misc API. kernel/module.c — loader. scripts/gdb/ — in-tree GDB helpers if you build vmlinux with scripts. Linux Device Drivers 3rd edition is old but the char-driver chapters still match the ideas (cdev numbers have changed; misc is easier now).

Windows readers: compare this ioctl path to a PASSIVE_LEVEL IRP_MJ_DEVICE_CONTROL that METHOD_BUFFERED copies into a kernel buffer, then a careless RtlCopyMemory into a too-small stack array. IRQL is the wrong panic button here; the Linux panic button is might_sleep and in_interrupt(). The overflow is the same class of C bug on both kernels.

E.3 Checklist before you say the lab works

- make produces vuln_lkm.ko and vuln_lkm_cli
- insmod prints g_val=100 in dmesg
- GET returns 100, cmd 0x80041b01
- SET 10, SUB 11, GET 4294967295, dmesg wrap=yes

- SET 0xfffffffffe, ADD 3, GET 1, dmesg wrap=yes
- QEMU boots with the gdb stub banner on port 1234
- GDB target remote works; after insmod, add-symbol-file binds four breaks
- STR hello logs len=5 into kbuf[32] and the guest lives
- You did not run a 40-byte STR on a host you need

E.4 Document history

Generated from the Vuln-LKM tree (github.com/mayday5000/Vuln-LKM) as an educational PDF. Sessions in chapters 8–10 follow the printk format and GDB commands in the source. They are expected lab output, not a dated capture from one machine. Rebuild this PDF if the source changes.

Companion PDFs live in Docs/: vuln_lkm.pdf, vuln_lkm_cli.pdf, vuln_lkm_run_debug.pdf. Markdown remain Docs/install-and-run.md, Docs/vulnerabilities.md, Docs/cli.md. If they drift, trust the .c files.

Appendix F. Integer overflow, continued

A wrap used as a size is the usual way unsigned overflow becomes a memory bug. Example pattern (not present in this module): `size = hdr->count * sizeof(*elt); ptr = kmalloc(size, GFP_KERNEL);` then a loop of `hdr->count` iterations. If `count` is a u32 near 2^{30} and `sizeof` is 16, the multiply wraps in 32-bit arithmetic before promotion, `kmalloc` gets a small size, the loop writes far past it. Mitigations: `check_mul_overflow`, `array_size()`, `struct_size()`, saturating helpers in `linux/overflow.h`, and never mixing a user u32 with a `size_t` multiply without a check.

Another pattern: `remaining = end - pos; if (remaining > user_len) remaining = user_len; copy.` If `pos > end` because both are unsigned and `pos` wrapped, `remaining` is huge. The lab's ADD/SUB are the arithmetic in isolation so you can see the number in GET before you meet it inside a size expression.

Comparison pitfalls: `if (off + len < off)` is a wrap check for unsigned `off+len`. Compilers may warn. `check_add_overflow(off, len, &sum)` is clearer. Signed `len < 0` checks are a different class (a negative length cast to `size_t` becomes huge). This lab's CLI `parse_u32` never yields a negative; the kernel side is u32 only.

`printf/printk %u` versus `%d`: printing a wrapped u32 with `%d` shows a negative if the value is $> \text{INT_MAX}$. The module uses `%u`. The CLI uses `%u` and `0x%08x` together so 4294967295 is not mistaken for -1 even though 0xffffffff is both.

F.1 Why g_val starts at 100

Zero is a poor demo default: GET 0 looks like an uninitialized buffer or a failed copy. 100 is visible in dmesg and in the CLI. SET exists so you do not need a wrap from 100 to hit the boundary; you plant 10 or 0xfffffffffe. Starting at `U32_MAX` would make the first ADD wrap immediately and confuse the 'control GET' step.

Appendix G. Buffer overflow, continued

C-string APIs (`strcpy`, `strcat`, `gets`, `sprintf`) are the textbook overflow tools. This module uses `memcpy` with an explicit length, which is the better API and still wrong if the length is not the destination size. The fix is `memcpy(kbuf, req.data, min(req.len, sizeof(kbuf)))` and then a separate NUL if you need a string — or, better, reject if `req.len > sizeof(kbuf)` with `-EINVAL` so the caller learns the limit. Silent truncation hides bugs in testers; reject is better for an ABI.

`strncpy` is not a fix: it does not always NUL-terminate, and it writes extra NULs. `strncpy` / `strncpy` (kernel) take the destination size. This lab's data is a counted buffer (`len + bytes`), not a C string, so counted `memcpy` with a `dest cap` is the matching API. The CLI sends C strings and sets `len=strlen`, which is why 40 A's is 40 bytes not 41.

Off-by-one: copying `n` bytes into a buffer of `n` is in-bounds for `memcpy`. Then writing `buf[n] = 0` is the extra byte. `vuln_lkm_str` does not add a NUL. A 32-byte copy is in-bounds. A 33-byte copy is not. The help uses 40 to stay away from

off-by-one arguments.

Kernel stack size is a hard limit. Recursion and large automatics are themselves overflows. struct vuln_lkm_str on stack (260 bytes) plus kbuf (32) plus padding is small. Do not 'fix' STR by placing char kbuf[256] on stack without thinking; 256 is still fine here, but the habit of huge stack buffers is how interrupt stacks die. A real fix would be a dest cap of 32 or a kmalloc with a hard max.

G.1 What this PDF will not diagram

No ROP chain, no canary leak recipe, no SMEP off switch, no cred overwrite. Those are exploit topics. The stack layout paragraph in Appendix B is so you can interpret GDB, not so you can aim a write. If a course asks you to 'exploit the module', that is a different document and not this repository.

Appendix H. Character devices, continued

Naming: /dev nodes are not unique by name across history. The pair (major, minor) is the kernel's identity. udev can create extra symlinks (disk/by-uuid, char/10:55). The CLI hardcodes /dev/vuln_lkm. If you mknod that name on the wrong major, ioctl will talk to another driver and you will get ENOTTY or worse. ls -l /dev/vuln_lkm should show major 10 (misc).

Permissions: mode 0666 is world-writable ioctls on a module that can oops the machine. That is acceptable only as a lab. A classroom VM should still not be shared with untrusted local users while the module is loaded. chmod 666 in load.sh races udev; if udev rewrites the node, you may need a udev rule. In QEMU busybox, udev is often absent and misc's mode sticks.

Blocking: this ioctl never waits on a waitqueue. It cannot hang except inside copy_*_user (page fault) or printk. A 'hung' CLI is usually GDB holding the vCPU at a breakpoint. continue in GDB. If the guest oopsed, the CLI is dead with it.

Signals: a Ctrl-C in the CLI while ioctl is in kernel will (normally) wait until the ioctl returns, unless the copy takes a signal and returns -EINTR. The module does not use restart_block. The CLI does not retry. Interactive mode's Ctrl-C usually kills the process; the fd closes; the module stays loaded.

Appendix I. ioctl packing workbook

Worked bits for GET. dir=_IOC_READ=2 → bits 31–30 = 10b → 0x80000000. size=4 → bits 29–16 = 4 → 0x00040000. type=0x1B → bits 15–8 → 0x00001b00. nr=1 → 0x00000001. OR = 0x80041b01. The CLI's first GET line should match.

Worked bits for STR. dir=_IOC_WRITE=1 → 0x40000000. size=260=0x104 → 0x01040000. type=0x1B → 0x00001b00. nr=5 → 5. OR = 0x41041b05. If you change VULN_LKM_STR_MAX, the size field changes, the command word changes, and an old CLI gets ENOTTY. That is why the header is shared.

_IOC_TYPECHECK exists so the third argument to _IOR is a type, not a number. Passing a sizeof accidentally can still compile in some trees and pack the wrong size. Our macros pass struct vuln_lkm_int and struct vuln_lkm_str.

Direction bits are documentation for tools (strace prints _IOC_READ/_WRITE) and for some generic copy layers. The kernel does not auto-copy based on dir for unlocked_ioctl; you still call copy_*_user. Some subsystems (USB, for example) do use the direction. This driver copies by hand in each case.

Appendix J. IRQL workbook for Windows readers

Map a hypothetical NT driver that did the same lab. A device object, a symbolic link \DosDevices\VulnLkm, `IRP_MJ_DEVICE_CONTROL` with `IOCTL` codes from `CTL_CODE(FILE_DEVICE_UNKNOWN, n, METHOD_BUFFERED, FILE_ANY_ACCESS)`. The I/O manager copies the input buffer to `Irp->AssociatedIrp.SystemBuffer` at `PASSIVE_LEVEL`. Your handler reads a `ULONG` and adds it to a static. That add still wraps. `METHOD_BUFFERED` already copied; you would not `ProbeForRead` the user VA in the common path. The stack smash would be a local `CHAR kbuf[32]` and `RtlCopyMemory(kbuf, src, len)` with `len` from the buffered header. `KeGetCurrentIrql()` would return `PASSIVE_LEVEL`. Raising to `DISPATCH_LEVEL` before that copy would be a bug if the source were still user memory; with `METHOD_BUFFERED` it is already kernel, but you still must not page.

Minifilter: a pre-create callback can run at `PASSIVE`. A post-callback depending on the operation might not. People stash work to a worker thread (`PASSIVE`) when they must touch paged memory. Linux `ioctl` does not need that stash because it is already a thread. Linux IRQs need a workqueue or a threaded interrupt for sleeping work. Different diagrams, same idea: sleeping work belongs in thread context.

`APC_LEVEL` has no Linux twin. Linux user APCs are not a first-class IRQL. Kernel preemption and `cond_resched` are the ways a task yields. Holding a spinlock suppresses that. Do not look for `KeAreApcsDisabled` in this module.

Appendix K. GDB and QEMU extras

info registers in x86_64 GDB: `rip` is the guest kernel instruction pointer. `rsp` is the kernel stack of the interrupted vCPU (the CLI's kernel stack when you break in `ioctl`). `orig_rax` is the syscall number if you stopped in syscall entry. For a break inside `vuln_lkm_add` you care about the C locals, not `rax`, unless you step in the add instruction itself (`add %reg, g_val`).

hardware breakpoints: `gdb hbreak` if software `int3` in module text is inconvenient. QEMU supports guest debug registers. Four hardware breaks is the x86 limit. We use four software breaks.

lx-symbols: if you load `vmlinux` and the kernel's gdb scripts, `lx-symbols` auto-loads module symbols from the guest. This repo's `debug_gdb.sh` does not depend on that; it uses `add-symbol-file`. If you have `vmlinux` matching the guest, `gdb vmlinux -x scripts/vuln_lkm.gdb` is nicer backtraces through `vfs_ioctl`.

QEMU monitor (`Ctrl-A C`): info registers, `x / 16gx addr, stop, cont`. You can dump guest memory without GDB. GDB is still the right tool for source-level breaks on `vuln_lkm_str`.

Multiple vCPUs: the script sets `-smp 1` so you do not chase a breakpoint on another core. If you raise `smp`, GDB's stub follows one thread; schedule-sensitive races on `g_val` would show up. The lab is not a race lab.

Appendix L. Build flags and debug info

`kbuild` uses the kernel's `CFLAGS`. Distro headers often build out-of-tree modules at `-O2` without `-g`. Then GDB has symbols but no lines. `EXTRA_CFLAGS='-g -fno-omit-frame-pointer'` on the make module line helps. Do not mix `-O0` with a kernel that was built at `-O2` for ABI reasons of your own; for this file `-O2 -g` is enough. The CLI is `-O2 -g-less` by default; `gdb` the CLI only if you are debugging `print_b64`, not the kernel.

`strip` on a `.ko` removes debug info and sometimes names. Do not strip the lab module. `objdump -d vuln_lkm.ko | less` shows `vuln_lkm_add`. `nm vuln_lkm.ko` shows `t vuln_lkm_add` (local text). A missing symbol means the compiler inlined it; the source marks the helpers `noinline-by-being-separate-functions` and the comments say GDB: break ... — if `gcc` inlines at high `-O`, add `noinline`. The committed Makefile does not pass `noinline`; if a break never hits, disassemble `vuln_lkm_ioctl` and see a `callq` or an inlined `add`.

Appendix M. WSL notes

WSL2 is a real Linux VM. insmod on WSL uses the Microsoft/WSL kernel. Headers must match that `uname -r`. Nested QEMU with KVM may not be available; TCG is slower and fine for this lab. Paths under `/mnt/c/Development/Git_Damian` are NTFS via drvfs; `kbuid` on drvfs can be fragile (case, execute bit, inotify). Building in the Linux filesystem (home) and copying the `.ko` is more reliable if `make` fails in odd ways.

The `9p` share in `run_qemu.sh` is the repo path on the Linux side. If that path is `/mnt/c/...`, QEMU still serves the files; performance is worse. A clone in `$HOME/Vuln-LKM` is the cleaner lab root.

Appendix N. printk format of every path

```
1 init: vuln_lkm: loaded, /dev/vuln_lkm, g_val=100
2 exit: vuln_lkm: unloaded
3 GET:  vuln_lkm: GET %u (0x%x)
4 SET:  vuln_lkm: SET %u -> %u          (old, new)
5 ADD:  vuln_lkm: ADD %u + %u = %u (wrap=%s)
6 SUB:  vuln_lkm: SUB %u - %u = %u (wrap=%s)
7 STR:  vuln_lkm: STR len=%u into kbuf[%u]
```

If you do not see these, you are looking at the wrong kernel log (another VM), or `rsyslog` filtered `KERN_INFO`, or you did not `insmod` this module (name collision with an older `.ko` still loaded — `load.sh rmmmods old names`).

Appendix O. End-to-end expected QEMU+GDB session (compressed)

```

1 # host terminal A
2 sudo ./scripts/install_qemu_gdb.sh
3 make
4 export KERNEL=$HOME/bzImage
5 ./scripts/run_qemu.sh
6
7 # QEMU serial (example)
8 Linux version ...
9 Kernel command line: console=ttyS0 nokaslr earlyprintk=serial
10 / # mkdir -p /mod && mount -t 9p -o trans=virtio modshare /mod
11 / # insmod /mod/vuln_lkm.ko
12 [ 12.40] vuln_lkm: loaded, /dev/vuln_lkm, g_val=100
13 / # cat /sys/module/vuln_lkm/sections/.text
14 0xffffffffc0002000
15
16 # host terminal B
17 ./scripts/debug_gdb.sh
18 (gdb) add-symbol-file vuln_lkm.ko 0xffffffffc0002000
19 (gdb) disable 1
20 (gdb) continue
21
22 # serial
23 / # /mod/vuln_lkm_cli set 0xfffffffffe
24 / # /mod/vuln_lkm_cli add 3
25
26 # gdb
27 Breakpoint 2, vuln_lkm_add (n=3) at vuln_lkm.c:17
28 (gdb) p g_val
29 $1 = 4294967294
30 (gdb) next
31 (gdb) next
32 (gdb) p g_val
33 $2 = 1
34 (gdb) continue
35
36 # serial
37 / # /mod/vuln_lkm_cli get
38 kernel value: 1 (0x00000001)
39 / # /mod/vuln_lkm_cli str hello
40 [ 20.01] vuln_lkm: STR len=5 into kbuf[32]
41
42 # gdb on STR
43 Breakpoint 4, vuln_lkm_str (arg=...) at vuln_lkm.c:41
44 (gdb) next
45 (gdb) p req.len
46 $3 = 5
47 (gdb) p sizeof(kbuf)
48 $4 = 32
49 (gdb) continue
50
51 # done
52 / # rmmod vuln_lkm
53 # QEMU: Ctrl-A X

```

That compressed session is the minimum complete debug run: wrap ADD under GDB, GET confirms 1, STR hello under GDB with len 5 versus 32. The 40-byte STR is optional and may end the guest.

Appendix P. Syscall path recap for the CLI

`open(2)` of `/dev/vuln_lkm` is `SYS_openat` on modern `x86_64` libc. The VFS looks up the dentry, checks mode, and `misc_open` stores the `miscdevice` in `file->private_data`. `ioctl(2)` is `SYS_ioctl`. The `cmd` word is passed in `rsi` after the `fd` in `rdi`; `arg` is `rdx`. `vuln_lkm_ioctl` switches on `cmd`. `GET` `copy_to_user` writes four bytes back into the CLI stack. The CLI then prints the value and base64. That is the whole round trip without QEMU-specific pieces.

If `GET` returns 100 but `dmesg` is empty, the kernel log level is below `info`. Raise it: `echo 8 > /proc/sys/kernel/printk`. In QEMU with `earlyprintk=serial` you should still see the module lines.

If GDB shows `cmd` truncated, you printed the wrong width. Use `p /x (unsigned int)cmd`. The full word is 32 bits. `0x80041b01` must not become `0x1b01`.

When finished, `rmmmod` then `ls /dev/vuln_lkm` should fail, `lsmod` should not list `vuln_lkm`, and `/sys/module/vuln_lkm` should be gone.

This appendix closes the sit-down read: theory, install, expected QEMU, expected GDB, full listings, glossary, failure table, and the syscall recap. The `.c` files remain the ABI.

Appendix Q. `copy_from_user` versus `memcpy`, one more time

`copy_from_user` can return a short count. `memcpy` cannot. After a successful copy of struct `vuln_lkm_str`, `req.data` is kernel memory. `memcpy(kbuf, req.data, req.len)` is then a kernel-to-kernel copy keyed off a user-chosen length. The first copy is bounded by `sizeof(req)`. The second is bounded by `req.len` after a cap against 256. `kbuf` is 32. Those three numbers are the string chapter of this lab.

`ADD` and `SUB` never copy more than four bytes. Their bug is the arithmetic, not the copy. `STR` copies 260 then 32-or-more. Do not mix the two bugs in your notes: `wrap` is `g_val`, `smash` is `kbuf`.